

컴퓨터 비전 프로젝트

YOLOv8 기반 제조 데이터 객체 탐지 실습

사전학습 모델 이해 → stamp 데이터 학습·평가 → 모델·조건별 성능 실험

작성자	hwang-ye-song
레포지토리	https://github.com/hwang-ye-song/CV_YOLO
작성일	2026-09-17
환경	Windows 11 · Python 3.11 · PyTorch 2.11 (CUDA 12.8) · Ultralytics 8.4 · RTX 5060 Laptop

목차

1. 실습 노트북 — yolov8_manufacturing_practice.ipynb

YOLOv8 기반 제조 데이터 객체 탐지 실습 [프로젝트]

항목	내용
목표	① 사전학습 YOLOv8의 추론 결과 구조를 이해한다 ② 제조 현장 데이터(stamp: 신발 + 도장)로 학습·평가한다 ③ 모델·데이터 조건을 바꿔 성능 범위를 실험한다
진행 방식	과제 기본 코드를 순서대로 따라가며 각 코드 아래에 의미를 정리 하고, 파트가 끝날 때마다 🧑🏻 개인 실험을 수행
환경	Windows 11 · Python 3.11 · PyTorch 2.11 (CUDA 12.8) · Ultralytics 8.4 · RTX 5060 Laptop 8GB
레포	https://github.com/hwang-ye-song/CV_YOLO

목차

- Part 1. YOLOv8 확인하기 (버스 이미지) → 🧑🏻 개인 실험 1
- Part 2. YOLOv8 학습 실습 (stamp 데이터셋) → 🧑🏻 개인 실험 2
- 목표 기준 평가 · 개인 회고 (KPT / AAR)

Part 1. YOLOv8 확인하기

```
In [1]: %pip install -q ultralytics
```

Note: you may need to restart the kernel to use updated packages.

📄 **설명** — YOLOv8을 만든 Ultralytics 패키지를 설치한다. 모델 정의·가중치 다운로드·학습·추론·평가 기능이 모두 들어 있다. (이 PC에서는 프로젝트 가상환경에 미리 설치돼 있어서 "이미 설치됨"으로 끝난다.)

Yolo 이해

```
In [2]: from ultralytics import YOLO
```

📄 **설명** — YOLO 클래스 하나로 탐지·분류·분할 등 모든 작업을 다룬다. 어떤 작업인지는 불러오는 가중치 파일이 정한다.

```
In [3]: model = YOLO('yolov8n.pt')
```

📄 **설명** — COCO 데이터셋(80개 클래스)으로 사전학습된 **YOLOv8 nano** 모델을 불러온다. 파일이 없으면 자동으로 받는다. **n** (nano) < **s** < **m** < **l** < **x** 순서로 커지고, 클수록 정확하지만 느리다.

```
In [4]: example_imgs = ['https://ultralytics.com/images/bus.jpg']
```

```
In [5]: from PIL import Image
import requests
from io import BytesIO
```

```
In [6]: response = requests.get(example_imgs[0])
img = Image.open(BytesIO(response.content))
```

```
In [7]: img.thumbnail((400,400))
img
```

Out[7]:



 **설명** — URL의 이미지를 메모리로 받아 PIL 이미지로 연다. `thumbnail` 은 원본 비율을 유지하면서 최대 400×400 안에 들어가게 줄인다. 화면 표시용일 뿐이고, 모델에는 원본 URL을 그대로 넘긴다.

```
In [8]: results = model.predict(example_imgs[0], save=True, conf=0.25)
```

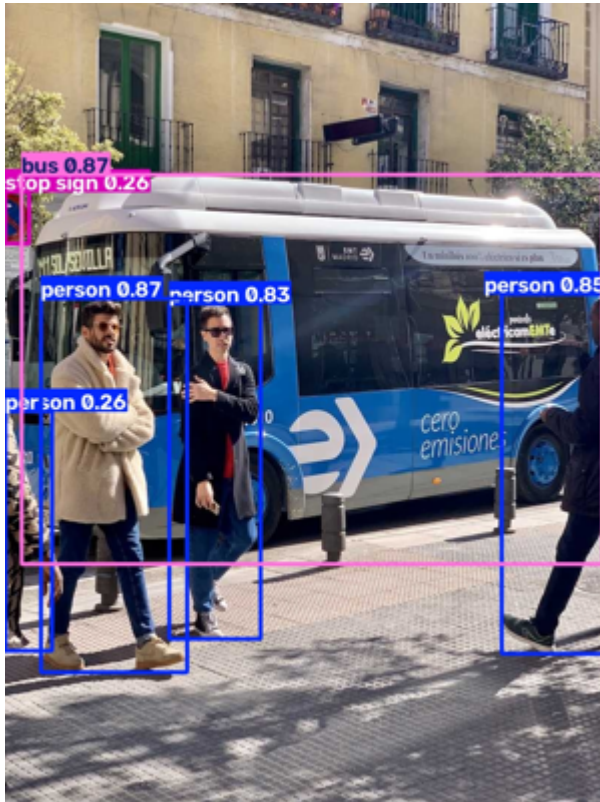
Found <https://ultralytics.com/images/bus.jpg> locally at bus.jpg
image 1/1 C:\Users\L0Q5060\Desktop\CV-YOLO\notebooks\bus.jpg: 640x480 4 persons, 1 bus, 1 stop sign, 12.1ms
Speed: 33.7ms preprocess, 12.1ms inference, 15.0ms postprocess per image at shape (1, 3, 640, 480)
Results saved to C:\Users\L0Q5060\runs\detect\predict

 **설명** — 추론을 실행한다.

- `save=True` : 박스를 그린 결과 이미지를 `runs/detect/predict*/` 에 저장
- `conf=0.25` : 신뢰도 0.25 미만인 박스는 버림
- 출력 로그의 `640x480` 은 모델 입력 크기다. 긴 변을 640에 맞추고 비율을 유지한다.

```
In [9]: output_img = Image.open(f'{results[0].save_dir}/bus.jpg')
output_img.thumbnail((400,400))
output_img
```

Out[9]:



 **설명** — 저장된 결과 이미지를 연다. 과제 코드는 `./runs/detect/predict/bus.jpg` 로 경로를 고정했지만, 여러 번 실행하면 `predict2`, `predict3` ... 로 폴더가 늘어난다. 그래서 결과 객체가 알려주는 `save_dir` 을 쓰도록 바꿨다.

In [10]: results

Out[10]: [ultralitics.engine.results.Results object with attributes:

```
boxes: ultralitics.engine.results.Boxes object
depth: None
keypoints: None
masks: None
names: {0: 'person', 1: 'bicycle', 2: 'car', 3: 'motorcycle', 4: 'airplane', 5:
'bus', 6: 'train', 7: 'truck', 8: 'boat', 9: 'traffic light', 10: 'fire hydran
t', 11: 'stop sign', 12: 'parking meter', 13: 'bench', 14: 'bird', 15: 'cat', 1
6: 'dog', 17: 'horse', 18: 'sheep', 19: 'cow', 20: 'elephant', 21: 'bear', 22:
'zebra', 23: 'giraffe', 24: 'backpack', 25: 'umbrella', 26: 'handbag', 27: 'ti
e', 28: 'suitcase', 29: 'frisbee', 30: 'skis', 31: 'snowboard', 32: 'sports bal
l', 33: 'kite', 34: 'baseball bat', 35: 'baseball glove', 36: 'skateboard', 37:
'surfboard', 38: 'tennis racket', 39: 'bottle', 40: 'wine glass', 41: 'cup', 42:
'fork', 43: 'knife', 44: 'spoon', 45: 'bowl', 46: 'banana', 47: 'apple', 48: 'sa
ndwich', 49: 'orange', 50: 'broccoli', 51: 'carrot', 52: 'hot dog', 53: 'pizza',
54: 'donut', 55: 'cake', 56: 'chair', 57: 'couch', 58: 'potted plant', 59: 'be
d', 60: 'dining table', 61: 'toilet', 62: 'tv', 63: 'laptop', 64: 'mouse', 65:
'remote', 66: 'keyboard', 67: 'cell phone', 68: 'microwave', 69: 'oven', 70: 'to
aster', 71: 'sink', 72: 'refrigerator', 73: 'book', 74: 'clock', 75: 'vase', 76:
'scissors', 77: 'teddy bear', 78: 'hair drier', 79: 'toothbrush'}
obb: None
orig_img: array([[119, 146, 172],
                 [121, 148, 174],
                 [122, 152, 177],
                 ...,
                 [161, 171, 188],
                 [160, 170, 187],
                 [160, 170, 187]],

                 [[120, 147, 173],
                 [122, 149, 175],
                 [123, 153, 178],
                 ...,
                 [161, 171, 188],
                 [160, 170, 187],
                 [160, 170, 187]],

                 [[123, 150, 176],
                 [124, 151, 177],
                 [125, 155, 180],
                 ...,
                 [161, 171, 188],
                 [160, 170, 187],
                 [160, 170, 187]],

                 ...,

                 [[183, 182, 186],
                 [179, 178, 182],
                 [180, 179, 183],
                 ...,
                 [121, 111, 117],
                 [113, 103, 109],
                 [115, 105, 111]],

                 [[165, 164, 168],
                 [173, 172, 176],
                 [187, 186, 190],
```

```

...,
[102, 92, 98],
[101, 91, 97],
[103, 93, 99]],

[[123, 122, 126],
[145, 144, 148],
[176, 175, 179],
...,
[ 95, 85, 91],
[ 96, 86, 92],
[ 98, 88, 94]]], shape=(1080, 810, 3), dtype=uint8)
orig_shape: (1080, 810)
path: 'C:\\Users\\LQ5060\\Desktop\\CV-YOLO\\notebooks\\bus.jpg'
probs: None
save_dir: 'C:\\Users\\LQ5060\\runs\\detect\\predict'
semantic_mask: None
speed: {'preprocess': 33.693799981847405, 'inference': 12.056499981554225, 'postprocess': 14.973999990615994}]

```

```
In [11]: type(results)
```

```
Out[11]: list
```

```
In [12]: type(results[0])
```

```
Out[12]: ultralytics.engine.results.Results
```

 **설명** — `predict()` 는 입력 이미지 한 장마다 **Results** 객체 하나를 담은 리스트를 돌려준다. 이미지가 한 장이라 `results[0]` 만 쓴다. **Results** 안에는 **boxes** (탐지), **masks** (분할), **keypoints** (자세), **probs** (분류), **names** (클래스 이름표)가 있고, 탐지 모델은 **boxes** 만 채워진다.

```
In [13]: results[0].boxes
```

Out[13]: ultralytics.engine.results.Boxes object with attributes:

```
cls: tensor([ 5.,  0.,  0.,  0.,  0., 11.], device='cuda:0')
conf: tensor([0.8732, 0.8657, 0.8528, 0.8253, 0.2608, 0.2552], device='cuda:0')
data: tensor([[2.2853e+01, 2.3128e+02, 8.0501e+02, 7.5686e+02, 8.7323e-01, 5.000
0e+00],
            [4.8546e+01, 3.9855e+02, 2.4534e+02, 9.0271e+02, 8.6572e-01, 0.0000e+0
0],
            [6.6947e+02, 3.9219e+02, 8.0972e+02, 8.7703e+02, 8.5275e-01, 0.0000e+0
0],
            [2.2151e+02, 4.0580e+02, 3.4497e+02, 8.5755e+02, 8.2528e-01, 0.0000e+0
0],
            [0.0000e+00, 5.5052e+02, 6.3039e+01, 8.7344e+02, 2.6076e-01, 0.0000e+0
0],
            [5.7797e-02, 2.5446e+02, 3.2557e+01, 3.2488e+02, 2.5521e-01, 1.1000e+0
1]], device='cuda:0')
id: None
is_track: False
orig_shape: (1080, 810)
shape: torch.Size([6, 6])
xywh: tensor([[413.9292, 494.0699, 782.1526, 525.5790],
            [146.9449, 650.6298, 196.7977, 504.1556],
            [739.5948, 634.6099, 140.2510, 484.8480],
            [283.2428, 631.6757, 123.4565, 451.7523],
            [ 31.5197, 711.9839,  63.0394, 322.9200],
            [ 16.3075, 289.6674,  32.4993,  70.4186]], device='cuda:0')
xywhn: tensor([[0.5110, 0.4575, 0.9656, 0.4866],
            [0.1814, 0.6024, 0.2430, 0.4668],
            [0.9131, 0.5876, 0.1731, 0.4489],
            [0.3497, 0.5849, 0.1524, 0.4183],
            [0.0389, 0.6592, 0.0778, 0.2990],
            [0.0201, 0.2682, 0.0401, 0.0652]], device='cuda:0')
xyxy: tensor([[2.2853e+01, 2.3128e+02, 8.0501e+02, 7.5686e+02],
            [4.8546e+01, 3.9855e+02, 2.4534e+02, 9.0271e+02],
            [6.6947e+02, 3.9219e+02, 8.0972e+02, 8.7703e+02],
            [2.2151e+02, 4.0580e+02, 3.4497e+02, 8.5755e+02],
            [0.0000e+00, 5.5052e+02, 6.3039e+01, 8.7344e+02],
            [5.7797e-02, 2.5446e+02, 3.2557e+01, 3.2488e+02]], device='cuda:0')
xyxyn: tensor([[2.8214e-02, 2.1415e-01, 9.9383e-01, 7.0080e-01],
            [5.9933e-02, 3.6903e-01, 3.0289e-01, 8.3584e-01],
            [8.2651e-01, 3.6314e-01, 9.9965e-01, 8.1207e-01],
            [2.7347e-01, 3.7574e-01, 4.2589e-01, 7.9403e-01],
            [0.0000e+00, 5.0974e-01, 7.7826e-02, 8.0874e-01],
            [7.1355e-05, 2.3561e-01, 4.0194e-02, 3.0081e-01]], device='cuda:0')
```

 설명 — Boxes 객체의 주요 속성

속성	내용
cls	클래스 번호 (5=bus, 0=person, 11=stop sign)
conf	신뢰도
xyxy / xywh	픽셀 좌표 (좌상단·우하단 / 중심·크기)
xyxyn / xywhn	위 좌표를 이미지 크기로 나눈 0~1 값
data	[x1, y1, x2, y2, conf, cls] 를 한 행에 모은 텐서

```
In [14]: url = "https://raw.githubusercontent.com/ultralytics/ultralytics/main/ultralytics/cfg/datasets/coco.yaml"
```

```
In [15]: import yaml
import requests

url = "https://raw.githubusercontent.com/ultralytics/ultralytics/main/ultralytics/cfg/datasets/coco.yaml"

response = requests.get(url)
data = yaml.safe_load(response.text)

print(data)
```

```
{'path': 'coco', 'train': 'train2017.txt', 'val': 'val2017.txt', 'test': 'test-dev
2017.txt', 'names': {0: 'person', 1: 'bicycle', 2: 'car', 3: 'motorcycle', 4: 'air
plane', 5: 'bus', 6: 'train', 7: 'truck', 8: 'boat', 9: 'traffic light', 10: 'fire
hydrant', 11: 'stop sign', 12: 'parking meter', 13: 'bench', 14: 'bird', 15: 'ca
t', 16: 'dog', 17: 'horse', 18: 'sheep', 19: 'cow', 20: 'elephant', 21: 'bear', 2
2: 'zebra', 23: 'giraffe', 24: 'backpack', 25: 'umbrella', 26: 'handbag', 27: 'ti
e', 28: 'suitcase', 29: 'frisbee', 30: 'skis', 31: 'snowboard', 32: 'sports ball',
33: 'kite', 34: 'baseball bat', 35: 'baseball glove', 36: 'skateboard', 37: 'surfb
oard', 38: 'tennis racket', 39: 'bottle', 40: 'wine glass', 41: 'cup', 42: 'fork',
43: 'knife', 44: 'spoon', 45: 'bowl', 46: 'banana', 47: 'apple', 48: 'sandwich', 4
9: 'orange', 50: 'broccoli', 51: 'carrot', 52: 'hot dog', 53: 'pizza', 54: 'donu
t', 55: 'cake', 56: 'chair', 57: 'couch', 58: 'potted plant', 59: 'bed', 60: 'dini
ng table', 61: 'toilet', 62: 'tv', 63: 'laptop', 64: 'mouse', 65: 'remote', 66: 'k
eyboard', 67: 'cell phone', 68: 'microwave', 69: 'oven', 70: 'toaster', 71: 'sin
k', 72: 'refrigerator', 73: 'book', 74: 'clock', 75: 'vase', 76: 'scissors', 77:
'teddy bear', 78: 'hair drier', 79: 'toothbrush'}, 'download': 'from pathlib impor
t Path\n\nfrom ultralytics.utils import ASSETS_URL\nfrom ultralytics.utils.downloa
ds import download\n\n# Download labels\nsegments = True # segment or box labels
\ndir = Path(yaml["path"]) # dataset root dir\nurls = [ASSETS_URL + ("/coco2017la
bels-segments.zip" if segments else "/coco2017labels.zip")] # labels\ndownload(ur
ls, dir=dir.parent)\n\n# Download data (test2017.zip excluded: ground truth is wit
hheld, only used for the eval-server test-dev split)\nurls = [\n    "http://image
s.cocodataset.org/zips/train2017.zip", # 19G, 118k images\n    "http://images.coc
odataset.org/zips/val2017.zip", # 1G, 5k images\n]\ndownload(urls, dir=dir / "ima
ges", threads=3)\n'}
```

 **설명** — COCO 데이터셋 설정 파일(yaml)을 읽는다. `names` 에 클래스 번호 → 이름 표가 있어서 `cls` 숫자를 사람이 읽을 수 있는 이름으로 바꿀 때 쓴다. (과제 원본의 `import requests=` 는 문법 오류라서 `import requests` 로 고쳤다.)

```
In [16]: results[0].boxes.conf
```

```
Out[16]: tensor([0.8732, 0.8657, 0.8528, 0.8253, 0.2608, 0.2552], device='cuda:0')
```

 **설명** — 6개 박스의 신뢰도. 버스·사람 3명은 0.8 이상으로 확실하고, 가장자리에 잘린 사람과 정지 표지판은 0.26 정도로 애매하다.

```
In [17]: results[0].boxes
```

Out[17]: ultralytics.engine.results.Boxes object with attributes:

```
cls: tensor([ 5.,  0.,  0.,  0.,  0., 11.], device='cuda:0')
conf: tensor([0.8732, 0.8657, 0.8528, 0.8253, 0.2608, 0.2552], device='cuda:0')
data: tensor([[2.2853e+01, 2.3128e+02, 8.0501e+02, 7.5686e+02, 8.7323e-01, 5.0000e+00],
              [4.8546e+01, 3.9855e+02, 2.4534e+02, 9.0271e+02, 8.6572e-01, 0.0000e+00],
              [6.6947e+02, 3.9219e+02, 8.0972e+02, 8.7703e+02, 8.5275e-01, 0.0000e+00],
              [2.2151e+02, 4.0580e+02, 3.4497e+02, 8.5755e+02, 8.2528e-01, 0.0000e+00],
              [0.0000e+00, 5.5052e+02, 6.3039e+01, 8.7344e+02, 2.6076e-01, 0.0000e+00],
              [5.7797e-02, 2.5446e+02, 3.2557e+01, 3.2488e+02, 2.5521e-01, 1.1000e+01]], device='cuda:0')
id: None
is_track: False
orig_shape: (1080, 810)
shape: torch.Size([6, 6])
xywh: tensor([[413.9292, 494.0699, 782.1526, 525.5790],
              [146.9449, 650.6298, 196.7977, 504.1556],
              [739.5948, 634.6099, 140.2510, 484.8480],
              [283.2428, 631.6757, 123.4565, 451.7523],
              [ 31.5197, 711.9839,  63.0394, 322.9200],
              [ 16.3075, 289.6674,  32.4993,  70.4186]], device='cuda:0')
xywhn: tensor([[0.5110, 0.4575, 0.9656, 0.4866],
               [0.1814, 0.6024, 0.2430, 0.4668],
               [0.9131, 0.5876, 0.1731, 0.4489],
               [0.3497, 0.5849, 0.1524, 0.4183],
               [0.0389, 0.6592, 0.0778, 0.2990],
               [0.0201, 0.2682, 0.0401, 0.0652]], device='cuda:0')
xyxy: tensor([[2.2853e+01, 2.3128e+02, 8.0501e+02, 7.5686e+02],
              [4.8546e+01, 3.9855e+02, 2.4534e+02, 9.0271e+02],
              [6.6947e+02, 3.9219e+02, 8.0972e+02, 8.7703e+02],
              [2.2151e+02, 4.0580e+02, 3.4497e+02, 8.5755e+02],
              [0.0000e+00, 5.5052e+02, 6.3039e+01, 8.7344e+02],
              [5.7797e-02, 2.5446e+02, 3.2557e+01, 3.2488e+02]], device='cuda:0')
xyxyn: tensor([[2.8214e-02, 2.1415e-01, 9.9383e-01, 7.0080e-01],
               [5.9933e-02, 3.6903e-01, 3.0289e-01, 8.3584e-01],
               [8.2651e-01, 3.6314e-01, 9.9965e-01, 8.1207e-01],
               [2.7347e-01, 3.7574e-01, 4.2589e-01, 7.9403e-01],
               [0.0000e+00, 5.0974e-01, 7.7826e-02, 8.0874e-01],
               [7.1355e-05, 2.3561e-01, 4.0194e-02, 3.0081e-01]], device='cuda:0')
```

In [18]: `img.size`

Out[18]: (300, 400)

 **설명** — `img` 는 위에서 `thumbnail` 로 줄였기 때문에 (300, 400) 이다. 박스 좌표는 원본(810×1080) 기준이므로, 정규화를 직접 계산하려면 원본 크기가 필요하다 → 아래에서 원본을 다시 받는다. PIL의 `size` 는 (너비, 높이) 순서라는 점도 주의.

In [19]: `results[0].boxes.xywh`

```
Out[19]: tensor([[413.9292, 494.0699, 782.1526, 525.5790],
                [146.9449, 650.6298, 196.7977, 504.1556],
                [739.5948, 634.6099, 140.2510, 484.8480],
                [283.2428, 631.6757, 123.4565, 451.7523],
                [ 31.5197, 711.9839,  63.0394, 322.9200],
                [ 16.3075, 289.6674,  32.4993,  70.4186]], device='cuda:0')
```

```
In [20]: results[0].boxes.xywhn
```

```
Out[20]: tensor([[0.5110, 0.4575, 0.9656, 0.4866],
                [0.1814, 0.6024, 0.2430, 0.4668],
                [0.9131, 0.5876, 0.1731, 0.4489],
                [0.3497, 0.5849, 0.1524, 0.4183],
                [0.0389, 0.6592, 0.0778, 0.2990],
                [0.0201, 0.2682, 0.0401, 0.0652]], device='cuda:0')
```

```
In [21]: import cv2
import requests
import numpy as np
from PIL import Image
from io import BytesIO

img_url = "https://ultralytics.com/images/bus.jpg"
response = requests.get(img_url)
img = Image.open(BytesIO(response.content)).convert("RGB")
img_np = np.array(img)
height, width = img_np.shape[:2]
```

```
In [22]: height, width
```

```
Out[22]: (1080, 810)
```

 **설명** — NumPy 배열의 `shape` 는 (높이, 너비, 채널) 순서다. 원본은 1080×810.

```
In [23]: bbox1 = results[0].boxes.xywh[0]
bbox1
```

```
Out[23]: tensor([413.9292, 494.0699, 782.1526, 525.5790], device='cuda:0')
```

```
In [24]: bbox1[0]/width, bbox1[1]/height, bbox1[2]/width, bbox1[3]/height
```

```
Out[24]: (tensor(0.5110, device='cuda:0'),
          tensor(0.4575, device='cuda:0'),
          tensor(0.9656, device='cuda:0'),
          tensor(0.4866, device='cuda:0'))
```

```
In [25]: results[0].boxes.xywhn
```

```
Out[25]: tensor([[0.5110, 0.4575, 0.9656, 0.4866],
                [0.1814, 0.6024, 0.2430, 0.4668],
                [0.9131, 0.5876, 0.1731, 0.4489],
                [0.3497, 0.5849, 0.1524, 0.4183],
                [0.0389, 0.6592, 0.0778, 0.2990],
                [0.0201, 0.2682, 0.0401, 0.0652]], device='cuda:0')
```

 **설명** — 첫 번째 박스(버스)의 `xywh` 를 원본 너비·높이로 직접 나눈 값이 `xywhn[0]` 과 정확히 같다. → **n 이 붙은 좌표 = 원본 크기로 나눈 비율**이다. 해상도와 무관해서 YOLO 학습 라벨(txt)도 이 `xywhn` 형식을 쓴다.

```
In [26]: bbox2 = results[0].boxes.xyxy[0]
         bbox2
```

```
Out[26]: tensor([ 22.8529, 231.2805, 805.0055, 756.8594], device='cuda:0')
```

```
In [27]: bbox2[0]/width, bbox2[1]/height, bbox2[2]/width, bbox2[3]/height
```

```
Out[27]: (tensor(0.0282, device='cuda:0'),
          tensor(0.2141, device='cuda:0'),
          tensor(0.9938, device='cuda:0'),
          tensor(0.7008, device='cuda:0'))
```

```
In [28]: results[0].boxes.xyxy[0]
```

```
Out[28]: tensor([0.0282, 0.2141, 0.9938, 0.7008], device='cuda:0')
```

 **설명** — `xyxy` 도 같은 방식으로 정규화하면 `xyxyn` 과 일치한다. `xyxy` 는 OpenCV 로 사각형을 그릴 때 그대로 쓸 수 있어서 시각화에 편하다.

시각화

```
In [29]: url = "https://ultralytics.com/images/bus.jpg"
         response = requests.get(url)
         img = np.array(Image.open(BytesIO(response.content)).convert("RGB"))
         h, w = img.shape[:2]
         img_copy = img.copy()
```

```
In [30]: xyxy_coordinate = results[0].boxes.xyxy.cpu().numpy()
```

```
In [31]: xyxy_coordinate
```

```
Out[31]: array([[ 22.853,   231.28,   805.01,   756.86],
                [  48.546,   398.55,   245.34,   902.71],
                [ 669.47,   392.19,   809.72,   877.03],
                [ 221.51,   405.8,   344.97,   857.55],
                [    0,   550.52,   63.039,   873.44],
                [ 0.057797,  254.46,   32.557,   324.88]], dtype=float32)
```

 **설명** — GPU에 있는 텐서를 `.cpu().numpy()` 로 NumPy 배열로 옮긴다. OpenCV·matplotlib은 GPU 텐서를 직접 다루지 못한다.

```
In [32]: import matplotlib.pyplot as plt

for box in xyxy_coordinate:
    x1, y1, x2, y2 = map(int, box)
    cv2.rectangle(img_copy, (x1, y1), (x2, y2), color=(255, 0, 0), thickness=2)

# 시각화
plt.figure(figsize=(10, 8))
plt.imshow(img_copy)
plt.title("YOLO xyxy Bounding Boxes")
plt.axis("off")
plt.show()
```



 **설명** — 모든 박스를 빨간 사각형으로 직접 그렸다. `cv2.rectangle` 은 정수 좌표가 필요해서 `int` 로 바꾼다. 이미지를 RGB로 읽었으므로 `(255, 0, 0)` 이 빨강이다. (OpenCV 기본인 BGR로 읽었다면 파랑이 된다.)

```
In [33]: results[0].boxes.data
```

```
Out[33]: tensor([[2.2853e+01, 2.3128e+02, 8.0501e+02, 7.5686e+02, 8.7323e-01, 5.0000e+00],
                [4.8546e+01, 3.9855e+02, 2.4534e+02, 9.0271e+02, 8.6572e-01, 0.0000e+00],
                [6.6947e+02, 3.9219e+02, 8.0972e+02, 8.7703e+02, 8.5275e-01, 0.0000e+00],
                [2.2151e+02, 4.0580e+02, 3.4497e+02, 8.5755e+02, 8.2528e-01, 0.0000e+00],
                [0.0000e+00, 5.5052e+02, 6.3039e+01, 8.7344e+02, 2.6076e-01, 0.0000e+00],
                [5.7797e-02, 2.5446e+02, 3.2557e+01, 3.2488e+02, 2.5521e-01, 1.1000e+01]], device='cuda:0')
```

```
In [34]: det = results[0].boxes.data.cpu().numpy()
```

```
In [35]: type(det)
```

```
Out[35]: numpy.ndarray
```

```
In [36]: url = "https://raw.githubusercontent.com/ultralytics/ultralytics/main/ultralytics/cfg/datasets/coco.yaml"

# URL에서 YAML 텍스트 불러오기
response = requests.get(url)
yaml_content = response.text

# YAML 파싱
parsed = yaml.safe_load(yaml_content)

# 클래스 이름 추출
class_names = parsed['names']
```

```
In [37]: class_names
```

```
Out[37]: {0: 'person',
 1: 'bicycle',
 2: 'car',
 3: 'motorcycle',
 4: 'airplane',
 5: 'bus',
 6: 'train',
 7: 'truck',
 8: 'boat',
 9: 'traffic light',
10: 'fire hydrant',
11: 'stop sign',
12: 'parking meter',
13: 'bench',
14: 'bird',
15: 'cat',
16: 'dog',
17: 'horse',
18: 'sheep',
19: 'cow',
20: 'elephant',
21: 'bear',
22: 'zebra',
23: 'giraffe',
24: 'backpack',
25: 'umbrella',
26: 'handbag',
27: 'tie',
28: 'suitcase',
29: 'frisbee',
30: 'skis',
31: 'snowboard',
32: 'sports ball',
33: 'kite',
34: 'baseball bat',
35: 'baseball glove',
36: 'skateboard',
37: 'surfboard',
38: 'tennis racket',
39: 'bottle',
40: 'wine glass',
41: 'cup',
42: 'fork',
43: 'knife',
44: 'spoon',
45: 'bowl',
46: 'banana',
47: 'apple',
48: 'sandwich',
49: 'orange',
50: 'broccoli',
51: 'carrot',
52: 'hot dog',
53: 'pizza',
54: 'donut',
55: 'cake',
56: 'chair',
57: 'couch',
58: 'potted plant',
```

```

59: 'bed',
60: 'dining table',
61: 'toilet',
62: 'tv',
63: 'laptop',
64: 'mouse',
65: 'remote',
66: 'keyboard',
67: 'cell phone',
68: 'microwave',
69: 'oven',
70: 'toaster',
71: 'sink',
72: 'refrigerator',
73: 'book',
74: 'clock',
75: 'vase',
76: 'scissors',
77: 'teddy bear',
78: 'hair drier',
79: 'toothbrush'}

```

데이터 클래스를 확인하였다 -> 시나리오 기반으로 생각할때는 필요없는 클래스들이 많
아보인다 => 액션

 **설명** — COCO 80개 클래스 중 제조 현장 시나리오에 필요한 것은 거의 없다. 필요한
클래스만 **걸러서** 쓰거나, 원하는 대상을 **새로 학습**해야 한다. 아래는 먼저 "걸러 쓰기"를
해본다.

```

In [38]: img_copy = img_np.copy()

for det in results[0].boxes.data:
    x1, y1, x2, y2, conf, cls = det.tolist()
    if int(cls) == 0:
        x1, y1, x2, y2 = map(int, [x1, y1, x2, y2])
        label = f"{class_names[int(cls)]} {conf:.2f}"
        cv2.rectangle(img_copy, (x1, y1), (x2, y2), (255, 0, 0), 2)
        cv2.putText(img_copy, label, (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX,
                    0.9, (255, 0, 0), 2)

plt.figure(figsize=(10, 8))
plt.imshow(img_copy)
plt.title("Only Class 0 (person) Bounding Boxes with Labels")
plt.axis("off")
plt.show()

```

Only Class 0 (person) Bounding Boxes with Labels



반쯤 가려진 사람도 잘 찾았다. 버스는 찾지않아도 되니까 결과 시각화에서 제외한다. 목표(시나리오)에 따르면 표지판에 대해서 찾을 수 있어야한다. -> 어떤 작업이 필요한가?

 **설명** — `boxes.data` 의 각 행을 `x1, y1, x2, y2, conf, cls` 로 풀고, `cls == 0` (person)인 박스만 이름·신뢰도와 함께 그렸다. 왼쪽 끝의 반쯤 잘린 사람도 0.26으로 잡혔다. 반면 표지판(stop sign)은 신뢰도가 낮고, COCO에 없는 표지판은 아예 찾을 수 없다. → 찾고 싶은 대상을 라벨링한 데이터로 추가 학습(파인튜닝) 이 필요하다 (Part 2).

```
In [39]: import os
bus_class_id = [k for k, v in class_names.items() if v == 'bus'][0]
os.makedirs("bus_crops", exist_ok=True)

count = 0
for det in results[0].boxes.data:
    x1, y1, x2, y2, conf, cls = det.tolist()
    if int(cls) == bus_class_id:
        x1, y1, x2, y2 = map(int, [x1, y1, x2, y2])
        cropped = img_np[y1:y2, x1:x2]
        save_path = f"bus_crops/bus_crop_{count}.jpg"
        cv2.imwrite(save_path, cv2.cvtColor(cropped, cv2.COLOR_RGB2BGR))
        print(f"버스 {count}번 잘라서 저장: {save_path}")
        count += 1
```

버스 0번 잘라서 저장: bus_crops/bus_crop_0.jpg

 **설명** — 클래스 이름에서 bus의 번호(5)를 찾아, 버스 박스 영역만 배열 슬라이싱(`img_np[y1:y2, x1:x2]`)으로 잘라 저장한다. `cv2.imwrite`는 BGR 순서로 저장하므로 RGB → BGR 변환이 필요하다. 제조 라인에서는 이렇게 탐지 → 잘라내기 → 정밀 검사의 2단계로 많이 쓴다.

```
In [40]: import os
import cv2
import matplotlib.pyplot as plt

crop_folder = "bus_crops"

image_files = sorted([f for f in os.listdir(crop_folder) if f.endswith(".jpg")])

num_images = len(image_files)
plt.figure(figsize=(5 * num_images, 5))

for i, filename in enumerate(image_files):
    img_path = os.path.join(crop_folder, filename)
    img = cv2.imread(img_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # OpenCV는 BGR로 불러오기 때문에 RGB로 변환

    plt.subplot(1, num_images, i + 1)
    plt.imshow(img)
    plt.title(filename)
    plt.axis("off")

plt.tight_layout()
plt.show()
```

bus_crop_0.jpg



 **설명** — 저장한 crop 이미지를 다시 읽어 확인한다. 버스 한 대가 잘려 나왔다.

개인 실험 1 — 사전학습 모델을 더 알아보기

기본 실습에서 생긴 질문을 직접 확인한다.

1. **신뢰도 임계값(conf)** 을 바꾸면 탐지 결과가 어떻게 달라지나?
2. **추가 샘플 이미지**에서도 잘 되나? **모델 크기·버전**(YOLOv8n / YOLOv8s / YOLO11n)에 따라 무엇이 달라지나?
3. **NMS의 IoU 임계값**은 겹친 박스에 어떤 영향을 주나?

```
In [41]: import pandas as pd
import torch
from pathlib import Path

DEVICE = 0 if torch.cuda.is_available() else "cpu"
bus_bgr = cv2.cvtColor(img_np, cv2.COLOR_RGB2BGR)

rows = []
for c in [0.05, 0.1, 0.25, 0.4, 0.55, 0.7, 0.85]:
    r = model.predict(bus_bgr, conf=c, device=DEVICE, verbose=False)[0]
    names = [class_names[int(k)] for k in r.boxes.cls]
    rows.append({"conf": c, "boxes": len(names), "person": names.count("person"),
                "bus": names.count("bus"), "others": ", ".join(n for n in names if n not in ("person", "bus")) or "-"},
                )
pd.DataFrame(rows)
```

Out[41]:

	conf	boxes	person	bus	others
0	0.05	13	5	3	stop sign, skateboard, bicycle, handbag, handbag
1	0.10	6	4	1	stop sign
2	0.25	6	4	1	stop sign
3	0.40	4	3	1	-
4	0.55	4	3	1	-
5	0.70	4	3	1	-
6	0.85	3	2	1	-

실험 1-1 결과 해석

- **0.05:** 박스가 13개로 늘었다. 사람 5명, 버스 3개(같은 버스에 박스가 겹쳐 나옴)에 skateboard-bicycle-handbag 같은 **없는 물체(오탐)** 가 섞였다.
- **0.10~0.25:** 6개(버스 1, 사람 4, 정지 표지판 1)로 안정적이다. 과제 기본값 0.25가 무난한 이유다.
- **0.40~0.70:** 4개. 신뢰도 0.26짜리였던 **왼쪽 끝의 잘린 사람**과 **정지 표지판**이 빠진다.
- **0.85:** 3개. 확실한 사람 한 명(0.83)까지 빠진다.

→ 임계값은 **농침(미탐)**과 **헛것(오탐)** 사이의 조절 손잡이다. 제조 검사처럼 놓치면 안 되는 상황에서는 낮게, 오탐 비용이 크면 높게 잡아야 한다.

In [42]:

```
SAMPLES = {"bus": "https://ultralytics.com/images/bus.jpg",
            "zidane": "https://ultralytics.com/images/zidane.jpg"}
MODELS = ["yolov8n.pt", "yolov8s.pt", "yolo11n.pt"]

samples = {k: cv2.imdecode(np.frombuffer(requests.get(u).content, np.uint8), cv
2.IMREAD_COLOR) for k, u in SAMPLES.items()}
rows, vis = [], {}
for mname in MODELS:
    m = YOLO(mname)
    for sname, im in samples.items():
        m.predict(im, device=DEVICE, verbose=False) # 첫 호출(워밍업)
        # 은 시간 측정에서 제외
        r = m.predict(im, conf=0.25, device=DEVICE, verbose=False)[0]
        names = [r.names[int(k)] for k in r.boxes.cls]
        rows.append({"model": mname, "image": sname, "boxes": len(names),
                    "detected": ", ".join(f"{n}x{names.count(n)}" for n in sorted(set(names))),
                    "mean_conf": round(float(r.boxes.conf.mean()), 3) if len(names)
                    else 0,
                    "infer_ms": round(r.speed["inference"], 1),
                    "params(M)": round(sum(p.numel() for p in m.model.parameters()) / 1e6, 1)})
        vis[(mname, sname)] = r.plot()[ :, :, :-1]
model_cmp = pd.DataFrame(rows)
model_cmp
```

Out[42]:

	model	image	boxes	detected	mean_conf	infer_ms	params(M)
0	yolov8n.pt	bus	6	bus×1, person×4, stop sign×1	0.655	15.8	3.2
1	yolov8n.pt	zidane	3	person×2, tie×1	0.649	13.1	3.2
2	yolov8s.pt	bus	5	bus×1, person×4	0.838	15.4	11.2
3	yolov8s.pt	zidane	3	person×2, tie×1	0.830	10.3	11.2
4	yolo11n.pt	bus	5	bus×1, person×4	0.837	18.2	2.6
5	yolo11n.pt	zidane	3	person×2, tie×1	0.691	14.7	2.6

In [43]:

```
fig, axes = plt.subplots(len(SAMPLES), len(MODELS), figsize=(5 * len(MODELS),
4.5 * len(SAMPLES)))
for i, sname in enumerate(SAMPLES):
    for j, mname in enumerate(MODELS):
        axes[i, j].imshow(vis[(mname, sname)]); axes[i, j].set_title(f"{mname}
| {sname}"); axes[i, j].axis("off")
plt.tight_layout(); plt.show()
```



실험 1-2 결과 해석

- 추가 샘플(zidane) 에서는 세 모델 모두 사람 2명 + 넥타이 1개로 같은 결과를 냈다.
- 버스 사진에서는 YOLOv8n만 정지 표지판(0.26)을 잡았고, YOLOv8s-YOLO11n은 버스 1 + 사람 4만 찾았다. 대신 평균 신뢰도가 v8n 0.66 → v8s-11n 0.84로 더 확신 있게 예측했다.
- 파라미터 수는 v8s(11.2M)가 v8n(3.2M)의 약 3.5배이고, YOLO11n(2.6M)은 v8n보다 오히려 작다.
- 한 장씩 추론한 시간(10~18ms)은 GPU 준비 시간의 영향이 커서, 이 실험만으로 속도 차이를 말하기는 어렵다. 속도 비교는 test 193장을 평균 낸 개인 실험 2-1에서 다시 본다.

→ 모델이 커지거나 최신 버전이라고 찾는 물체 수가 늘지는 않았고, 같은 물체를 더 확신 있게 맞히는 쪽으로 차이가 났다.

```
In [44]: rows = []
for iou in [0.3, 0.5, 0.7, 0.9]:
    for sname, im in samples.items():
        r = model.predict(im, conf=0.1, iou=iou, device=DEVICE, verbose=False)
        rows.append({"iou(NMS)": iou, "image": sname, "boxes": len(r.boxes)})
pd.DataFrame(rows).pivot(index="iou(NMS)", columns="image", values="boxes")
```

```
Out[44]:
```

	image	bus	zidane
iou(NMS)			
0.3	6	6	
0.5	6	8	
0.7	6	12	
0.9	9	13	

 **설명** — NMS(Non-Maximum Suppression)는 같은 물체에 겹쳐 나온 박스 중 신뢰도가 가장 높은 것만 남긴다. iou 값보다 많이 겹치는 박스를 지우기 때문에, 값이 클수록 겹친 박스를 더 많이 살려 둔다.

실험 1-3 결과 해석

- 버스 사진은 IoU 0.3~0.7에서 6개로 그대로였고, 0.9에서야 9개로 늘었다. 사람들이 서로 많이 겹치지 않아서다.
- zidane 사진은 0.3 → 6개, 0.5 → 8개, 0.7 → 12개, 0.9 → 13개로 크게 변했다. 같은 사람에게 여러 박스가 겹쳐 나오는데, IoU 기준이 느슨할수록 그 중복 박스가 지워지지 않고 남는다.

→ 제품이 촘촘히 붙어 있는 라인에서는 IoU를 너무 낮추면 붙은 제품 두 개가 하나로 합쳐지고, 너무 높이면 한 제품에 박스가 여러 개 생긴다. 기본값 0.7 근처에서 현장 배치에 맞게 조정해야 한다.

Part 2. YOLOv8 학습 실습

데이터셋 다운로드

<https://universe.roboflow.com/warisara-kaewsuwan-cf2hs/stamp-bcrhe>

- 사용 버전: **v10** (2022-04-02) · YOLOv8 형식 · 클래스 `shoes`, `stamp`
- 분할: train 1,100 / valid 273 / test 193 · 416×416으로 리사이즈됨
- 시나리오: 작업대 위에서 신발에 **도장(stamp)** 이 찍혔는지 위에서 내려다보며 확인하는 제조 공정 영상

```
In [45]: from ultralytics import YOLO
import os

model = YOLO("yolov8n.pt")
```

- datasets
 - stamp
 - test
 - train
 - valid
 - data.yaml

 **설명** — 이 PC에서는 `CV-YOLO/data/stamp/` 아래에 같은 구조로 두었다. Roboflow의 `data.yaml` 은 `../train/images` 같은 상대경로라서 실행 위치에 따라 깨진다 → 절대경로 버전을 만들어 쓴다.

```
In [46]: ROOT = Path.cwd().parent
STAMP = ROOT / "data" / "stamp"
RUNS = ROOT / "runs" / "stamp"

cfg = yaml.safe_load((STAMP / "data.yaml").read_text(encoding="utf-8"))
DATA = STAMP / "data_local.yaml"
DATA.write_text(yaml.safe_dump({"path": STAMP.as_posix(), "train": "train/images",
                                "val": "valid/images",
                                "test": "test/images", "names": cfg["names"]}, allow_unicode=True), encoding="utf-8")
print(DATA.read_text(encoding="utf-8"))
```

```
names:
- shoes
- stamp
path: C:/Users/LOQ5060/Desktop/CV-YOLO/data/stamp
test: test/images
train: train/images
val: valid/images
```

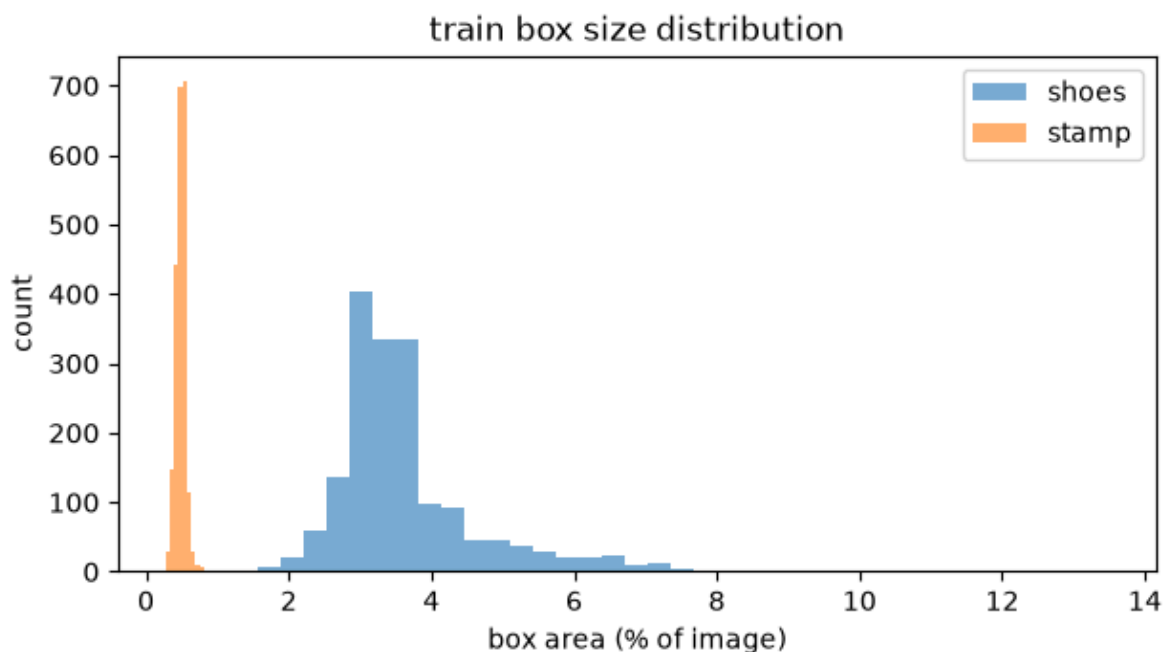
In [47]: # 학습 전에 데이터 분포부터 확인

```
def split_stats(split):
    labels = list((STAMP / split / "labels").glob("*.txt"))
    cls = [int(l.split()[0]) for f in labels for l in f.read_text().splitlines() if l.strip()]
    sizes = [float(l.split()[3]) * float(l.split()[4]) for f in labels for l in f.read_text().splitlines() if l.strip()]
    return {"split": split, "images": len(labels), "shoes": cls.count(0), "stamp": cls.count(1),
            "평균 박스 면적(%)": round(100 * np.mean(sizes), 2)}

def box_area(split, c):
    return [100 * float(l.split()[3]) * float(l.split()[4])
            for f in (STAMP / split / "labels").glob("*.txt") for l in f.read_text().splitlines() if l.strip() and int(l.split()[0]) == c]

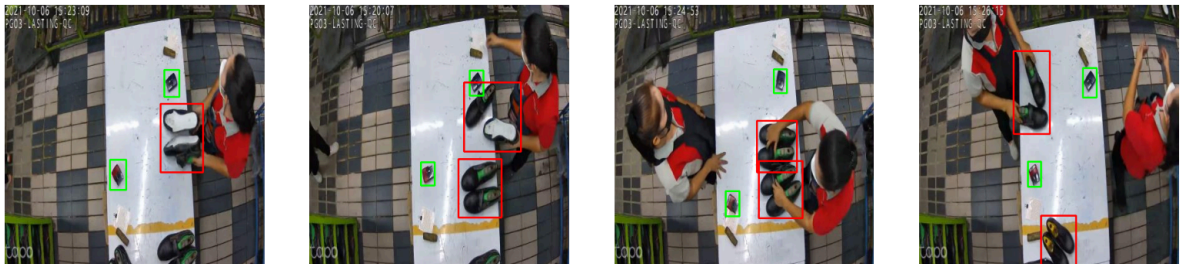
display(pd.DataFrame([split_stats(s) for s in ("train", "valid", "test")]))
plt.figure(figsize=(7, 3.5))
plt.hist(box_area("train", 0), bins=40, alpha=.6, label="shoes"); plt.hist(box_area("train", 1), bins=40, alpha=.6, label="stamp")
plt.xlabel("box area (% of image)"); plt.ylabel("count"); plt.legend(); plt.title("train box size distribution"); plt.show()
```

	split	images	shoes	stamp	평균 박스 면적(%)
0	train	1100	1750	2181	1.90
1	valid	273	408	544	1.59
2	test	193	250	383	1.49



```
In [48]: samples_tr = sorted((STAMP / "train" / "images").glob("*"))[:275][:4]
fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for ax, p in zip(axes, samples_tr):
    im = cv2.cvtColor(cv2.imread(str(p)), cv2.COLOR_BGR2RGB); H, W = im.shape[:2]
    for line in (STAMP / "train" / "labels" / (p.stem + ".txt")).read_text().splitlines():
        c, cx, cy, bw, bh = map(float, line.split()[:5])
        cv2.rectangle(im, (int((cx-bw/2)*W), int((cy-bh/2)*H)), (int((cx+bw/2)*W), int((cy+bh/2)*H)),
            (255, 0, 0) if int(c) == 0 else (0, 255, 0), 2)
    ax.imshow(im); ax.axis("off")
plt.suptitle("train labels (red = shoes, green = stamp)"); plt.tight_layout();
plt.show()
```

train labels (red = shoes, green = stamp)



데이터 확인 결과

- train 1,100장에 **shoes 1,750개 / stamp 2,181개**로 두 클래스가 크게 치우치지 않았다. 사진 한 장에 도장이 평균 2개쯤 있다.
- 박스 하나가 이미지에서 차지하는 면적이 평균 **1.5~1.9%**로 매우 작다. 히스토그램을 보면 **stamp가 shoes보다 훨씬 작은 쪽에 몰려 있다**. 416×416 이미지에서 도장은 한 번이 20픽셀 안팎이다.
- 라벨 샘플을 보면 위에서 내려다본 작업대 영상이고, 작업자의 손·팔이 신발과 도장을 자주 가린다.

→ 예상: 작은 도장은 위치를 대충 맞추기는 쉽지만 박스를 딱 맞추기는 어렵고(mAP50-95 낮음), 가려진 신발은 놓치기 쉬울 것이다.

```
In [49]: model.train(
    data=str(DATA),
    epochs=20,
    imgsz=640,
    batch=16,
    name="stamp_detect",
    device=0,
    project=str(RUNS), exist_ok=True, workers=2, seed=42
)
```

Ultralytics 8.4.154 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU, 8151MiB)

engine\trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugument, batch=16, bgr=0.0, box=7.5, cache=False, cfg=None, channels_last=None, classes=None, close_mosaic=10, cls=0.5, cls_pw=0.0, cls_remap=True, compile=False, conf=None, copy_paste=0.0, copy_paste_mode=flip, cos_lr=False, cutmix=0.0, data=C:\Users\LOQ5060\Desktop\CV-YOLO\data\stamp\data_local.yaml, degrees=0.0, deterministic=True, device=0, dfl=1.5, dgrad=0.5, dis=6.0, distill_model=None, dlam=1.0, dlog=1.0, dnn=False, dropout=0.0, dynamic=False, embed=None, epochs=20, erasing=0.4, exist_ok=True, flipplr=0.5, flipud=0.0, format=torchscript, fraction=1.0, freeze=None, hsv_h=0.015, hsv_s=0.7, hsv_v=0.4, imgsz=640, iou=0.7, keras=False, kobj=1.0, line_width=None, lr0=0.01, lrf=0.01, mask_ratio=4, max_det=300, mixup=0.0, mode=train, model=yolov8n.pt, momentum=0.937, mosaic=1.0, multi_scale=0.0, name=stamp_detect, nbs=64, nms=None, opset=None, optimize=False, optimizer=auto, overlap_mask=True, patience=100, perspective=0.0, plots=True, pose=12.0, pretrained=True, profile=False, project=C:\Users\LOQ5060\Desktop\CV-YOLO\runs\stamp, quantize=None, rect=False, resume=False, retina_masks=False, rle=1.0, save=True, save_conf=False, save_crop=False, save_dir=C:\Users\LOQ5060\Desktop\CV-YOLO\runs\stamp\stamp_detect, save_frames=False, save_json=False, save_period=-1, save_txt=False, scale=0.5, seed=42, shear=0.0, show=False, show_boxes=True, show_conf=True, show_labels=True, simplify=True, single_cls=False, source=None, split=val, stream_buffer=False, task=detect, time=None, tracker=tracktrack.yaml, translate=0.1, val=True, verbose=True, vid_stride=1, visualize=False, warmup_bias_lr=0.1, warmup_epochs=3.0, warmup_momentum=0.8, weight_decay=0.0005, workers=2, workspace=None

Overriding model.yaml nc=80 with nc=2

	from	n	params	module
arguments				
0	-1	1	464	ultralytics.nn.modules.conv.Conv
[3, 16, 3, 2]				
1	-1	1	4672	ultralytics.nn.modules.conv.Conv
[16, 32, 3, 2]				
2	-1	1	7360	ultralytics.nn.modules.block.C2f
[32, 32, 1, True]				
3	-1	1	18560	ultralytics.nn.modules.conv.Conv
[32, 64, 3, 2]				
4	-1	2	49664	ultralytics.nn.modules.block.C2f
[64, 64, 2, True]				
5	-1	1	73984	ultralytics.nn.modules.conv.Conv
[64, 128, 3, 2]				
6	-1	2	197632	ultralytics.nn.modules.block.C2f
[128, 128, 2, True]				
7	-1	1	295424	ultralytics.nn.modules.conv.Conv
[128, 256, 3, 2]				
8	-1	1	460288	ultralytics.nn.modules.block.C2f
[256, 256, 1, True]				
9	-1	1	164608	ultralytics.nn.modules.block.SPPF
[256, 256, 5]				
... (142줄 생략) ...				
	all		273	952
				0.953
				0.964
				0.985
0.62				

20 epochs completed in 0.055 hours.

Optimizer stripped from C:\Users\LOQ5060\Desktop\CV-YOLO\runs\stamp\stamp_detect\weights\last.pt, 6.2MB

Optimizer stripped from C:\Users\LOQ5060\Desktop\CV-YOLO\runs\stamp\stamp_detect\weights\best.pt, 6.2MB

Validating C:\Users\LOQ5060\Desktop\CV-YOLO\runs\stamp\stamp_detect\weights\best.p

t...

Ultralytics 8.4.154 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU, 8151MiB)

Model summary (fused): 72 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs

	all	273	952	0.941	0.971	0.984
--	-----	-----	-----	-------	-------	-------

0.642

	shoes	204	408	0.944	0.944	0.972
--	-------	-----	-----	-------	-------	-------

0.712

	stamp	273	544	0.939	0.998	0.995
--	-------	-----	-----	-------	-------	-------

0.573

Speed: 0.3ms preprocess, 1.6ms inference, 0.0ms loss, 1.4ms postprocess per image

Results saved to C:\Users\LOQ5060\Desktop\CV-YOLO\runs\stamp\stamp_detect

Out[49]: ultralytics.utils.metrics.DetMetrics object with attributes:

```
ap_class_index: array([0, 1])
box: ultralytics.utils.metrics.Metric object
confusion_matrix: <ultralytics.utils.metrics.ConfusionMatrix object at 0x00000240939E5110>
curves: ['Precision-Recall(B)', 'F1-Confidence(B)', 'Precision-Confidence(B)', 'Recall-Confidence(B)']
curves_results: [[array([
0, 0.001001, 0.002002, 0.003003,
0.004004, 0.005005, 0.006006, 0.007007, 0.008008, 0.009009,
0.01001, 0.011011, 0.012012, 0.013013, 0.014014, 0.015015, 0.0
16016, 0.017017, 0.018018, 0.019019, 0.02002, 0.021021, 0.022
022, 0.023023,
0.024024, 0.025025, 0.026026, 0.027027, 0.028028, 0.029
029, 0.03003, 0.031031, 0.032032, 0.033033, 0.034034, 0.03503
5, 0.036036, 0.037037, 0.038038, 0.039039, 0.04004, 0.041041,
0.042042, 0.043043, 0.044044, 0.045045, 0.046046, 0.047047,
0.048048, 0.049049, 0.05005, 0.051051, 0.052052, 0.053
053, 0.054054, 0.055055, 0.056056, 0.057057, 0.058058, 0.05905
9, 0.06006, 0.061061, 0.062062, 0.063063, 0.064064, 0.065065,
0.066066, 0.067067, 0.068068, 0.069069, 0.07007, 0.071071,
0.072072, 0.073073, 0.074074, 0.075075, 0.076076, 0.077
077, 0.078078, 0.079079, 0.08008, 0.081081, 0.082082, 0.08308
3, 0.084084, 0.085085, 0.086086, 0.087087, 0.088088, 0.089089,
0.09009, 0.091091, 0.092092, 0.093093, 0.094094, 0.095095,
0.096096, 0.097097, 0.098098, 0.099099, 0.1001, 0.1
011, 0.1021, 0.1031, 0.1041, 0.10511, 0.10611, 0.1071
1, 0.10811, 0.10911, 0.11011, 0.11111, 0.11211, 0.11311,
0.11411, 0.11512, 0.11612, 0.11712, 0.11812, 0.11912,
0.12012, 0.12112, 0.12212, 0.12312, 0.12412, 0.12
513, 0.12613, 0.12713, 0.12813, 0.12913, 0.13013, 0.1311
3, 0.13213, 0.13313, 0.13413, 0.13514, 0.13614, 0.13714,
0.13814, 0.13914, 0.14014, 0.14114, 0.14214, 0.14314,
0.14414, 0.14515, 0.14615, 0.14715, 0.14815, 0.14
915, 0.15015, 0.15115, 0.15215, 0.15315, 0.15415, 0.1551
6, 0.15616, 0.15716, 0.15816, 0.15916, 0.16016, 0.16116,
0.16216, 0.16316, 0.16416, 0.16517, 0.16617, 0.16717,
0.16817, 0.16917, 0.17017, 0.17117, 0.17217, 0.17
317, 0.17417, 0.17518, 0.17618, 0.17718, 0.17818, 0.1791
8, 0.18018, 0.18118, 0.18218, 0.18318, 0.18418, 0.18519,
0.18619, 0.18719, 0.18819, 0.18919, 0.19019, 0.19119,
0.19219, 0.19319, 0.19419, 0.1952, 0.1962, 0.1
972, 0.1982, 0.1992, 0.2002, 0.2012, 0.2022, 0.203
2, 0.2042, 0.20521, 0.20621, 0.20721, 0.20821, 0.20921,
0.21021, 0.21121, 0.21221, 0.21321, 0.21421, 0.21522,
0.21622, 0.21722, 0.21822, 0.21922, 0.22022, 0.22
122, 0.22222, 0.22322, 0.22422, 0.22523, 0.22623, 0.2272
3, 0.22823, 0.22923, 0.23023, 0.23123, 0.23223, 0.23323,
0.23423, 0.23524, 0.23624, 0.23724, 0.23824, 0.23924,
0.24024, 0.24124, 0.24224, 0.24324, 0.24424, 0.24
525, 0.24625, 0.24725, 0.24825, 0.24925, 0.25025, 0.2512
5, 0.25225, 0.25325, 0.25425, 0.25526, 0.25626, 0.25726,
0.25826, 0.25926, 0.26026, 0.26126, 0.26226, 0.26326,
0.26426, 0.26527, 0.26627, 0.26727, 0.26827, 0.26
927, 0.27027, 0.27127, 0.27227, 0.27327, 0.27427, 0.2752
8, 0.27628, 0.27728, 0.27828, 0.27928, 0.28028, 0.28128,
0.28228, 0.28328, 0.28428, 0.28529, 0.28629, 0.28729,
0.28829, 0.28929, 0.29029, 0.29129, 0.29229, 0.29
329, 0.29429, 0.2953, 0.2963, 0.2973, 0.2983, 0.299
```

3, 0.3003, 0.3013, 0.3023, 0.3033, 0.3043, 0.30531,
0.30631, 0.30731, 0.30831, 0.30931, 0.31031, 0.31131,
0.31231, 0.31331, 0.31431, 0.31532, 0.31632, 0.31
732, 0.31832, 0.31932, 0.32032, 0.32132, 0.32232, 0.3233
2, 0.32432, 0.32533, 0.32633, 0.32733, 0.32833, 0.32933,
0.33033, 0.33133, 0.33233, 0.33333, 0.33433, 0.33534,
0.33634, 0.33734, 0.33834, 0.33934, 0.34034, 0.34
134, 0.34234, 0.34334, 0.34434, 0.34535, 0.34635, 0.3473
5, 0.34835, 0.34935, 0.35035, 0.35135, 0.35235, 0.35335,
0.35435, 0.35536, 0.35636, 0.35736, 0.35836, 0.35936,
0.36036, 0.36136, 0.36236, 0.36336, 0.36436, 0.36
537, 0.36637, 0.36737, 0.36837, 0.36937, 0.37037, 0.3713
7, 0.37237, 0.37337, 0.37437, 0.37538, 0.37638, 0.37738,
0.37838, 0.37938, 0.38038, 0.38138, 0.38238, 0.38338,
0.38438, 0.38539, 0.38639, 0.38739, 0.38839, 0.38
939, 0.39039, 0.39139, 0.39239, 0.39339, 0.39439, 0.395
4, 0.3964, 0.3974, 0.3984, 0.3994, 0.4004, 0.4014,
0.4024, 0.4034, 0.4044, 0.40541, 0.40641, 0.40741,
0.40841, 0.40941, 0.41041, 0.41141, 0.41241, 0.41
341, 0.41441, 0.41542, 0.41642, 0.41742, 0.41842, 0.4194
2, 0.42042, 0.42142, 0.42242, 0.42342, 0.42442, 0.42543,
0.42643, 0.42743, 0.42843, 0.42943, 0.43043, 0.43143,
0.43243, 0.43343, 0.43443, 0.43544, 0.43644, 0.43
744, 0.43844, 0.43944, 0.44044, 0.44144, 0.44244, 0.4434
4, 0.44444, 0.44545, 0.44645, 0.44745, 0.44845, 0.44945,
0.45045, 0.45145, 0.45245, 0.45345, 0.45445, 0.45546,
0.45646, 0.45746, 0.45846, 0.45946, 0.46046, 0.46
146, 0.46246, 0.46346, 0.46446, 0.46547, 0.46647, 0.4674
7, 0.46847, 0.46947, 0.47047, 0.47147, 0.47247, 0.47347,
0.47447, 0.47548, 0.47648, 0.47748, 0.47848, 0.47948,
0.48048, 0.48148, 0.48248, 0.48348, 0.48448, 0.48
549, 0.48649, 0.48749, 0.48849, 0.48949, 0.49049, 0.4914
9, 0.49249, 0.49349, 0.49449, 0.4955, 0.4965, 0.4975,
0.4985, 0.4995, 0.5005, 0.5015, 0.5025, 0.5035,
0.5045, 0.50551, 0.50651, 0.50751, 0.50851, 0.50
951, 0.51051, 0.51151, 0.51251, 0.51351, 0.51451, 0.5155
2, 0.51652, 0.51752, 0.51852, 0.51952, 0.52052, 0.52152,
0.52252, 0.52352, 0.52452, 0.52553, 0.52653, 0.52753,
0.52853, 0.52953, 0.53053, 0.53153, 0.53253, 0.53
353, 0.53453, 0.53554, 0.53654, 0.53754, 0.53854, 0.5395
4, 0.54054, 0.54154, 0.54254, 0.54354, 0.54454, 0.54555,
0.54655, 0.54755, 0.54855, 0.54955, 0.55055, 0.55155,
0.55255, 0.55355, 0.55455, 0.55556, 0.55656, 0.55
756, 0.55856, 0.55956, 0.56056, 0.56156, 0.56256, 0.5635
6, 0.56456, 0.56557, 0.56657, 0.56757, 0.56857, 0.56957,
0.57057, 0.57157, 0.57257, 0.57357, 0.57457, 0.57558,
0.57658, 0.57758, 0.57858, 0.57958, 0.58058, 0.58
158, 0.58258, 0.58358, 0.58458, 0.58559, 0.58659, 0.5875
9, 0.58859, 0.58959, 0.59059, 0.59159, 0.59259, 0.59359,
0.59459, 0.5956, 0.5966, 0.5976, 0.5986, 0.5996,
0.6006, 0.6016, 0.6026, 0.6036, 0.6046, 0.60
561, 0.60661, 0.60761, 0.60861, 0.60961, 0.61061, 0.6116
1, 0.61261, 0.61361, 0.61461, 0.61562, 0.61662, 0.61762,
0.61862, 0.61962, 0.62062, 0.62162, 0.62262, 0.62362,
0.62462, 0.62563, 0.62663, 0.62763, 0.62863, 0.62
963, 0.63063, 0.63163, 0.63263, 0.63363, 0.63463, 0.6356
4, 0.63664, 0.63764, 0.63864, 0.63964, 0.64064, 0.64164,
0.64264, 0.64364, 0.64464, 0.64565, 0.64665, 0.64765,
0.64865, 0.64965, 0.65065, 0.65165, 0.65265, 0.65

365, 0.65465, 0.65566, 0.65666, 0.65766, 0.65866, 0.6596
6, 0.66066, 0.66166, 0.66266, 0.66366, 0.66466, 0.66567,
0.66667, 0.66767, 0.66867, 0.66967, 0.67067, 0.67167,
0.67267, 0.67367, 0.67467, 0.67568, 0.67668, 0.67
768, 0.67868, 0.67968, 0.68068, 0.68168, 0.68268, 0.6836
8, 0.68468, 0.68569, 0.68669, 0.68769, 0.68869, 0.68969,
0.69069, 0.69169, 0.69269, 0.69369, 0.69469, 0.6957,
0.6967, 0.6977, 0.6987, 0.6997, 0.7007, 0.7
017, 0.7027, 0.7037, 0.7047, 0.70571, 0.70671, 0.7077
1, 0.70871, 0.70971, 0.71071, 0.71171, 0.71271, 0.71371,
0.71471, 0.71572, 0.71672, 0.71772, 0.71872, 0.71972,
0.72072, 0.72172, 0.72272, 0.72372, 0.72472, 0.72
573, 0.72673, 0.72773, 0.72873, 0.72973, 0.73073, 0.7317
3, 0.73273, 0.73373, 0.73473, 0.73574, 0.73674, 0.73774,
0.73874, 0.73974, 0.74074, 0.74174, 0.74274, 0.74374,
0.74474, 0.74575, 0.74675, 0.74775, 0.74875, 0.74
975, 0.75075, 0.75175, 0.75275, 0.75375, 0.75475, 0.7557
6, 0.75676, 0.75776, 0.75876, 0.75976, 0.76076, 0.76176,
0.76276, 0.76376, 0.76476, 0.76577, 0.76677, 0.76777,
0.76877, 0.76977, 0.77077, 0.77177, 0.77277, 0.77
377, 0.77477, 0.77578, 0.77678, 0.77778, 0.77878, 0.7797
8, 0.78078, 0.78178, 0.78278, 0.78378, 0.78478, 0.78579,
0.78679, 0.78779, 0.78879, 0.78979, 0.79079, 0.79179,
0.79279, 0.79379, 0.79479, 0.7958, 0.7968, 0.7
978, 0.7988, 0.7998, 0.8008, 0.8018, 0.8028, 0.803
8, 0.8048, 0.80581, 0.80681, 0.80781, 0.80881, 0.80981,
0.81081, 0.81181, 0.81281, 0.81381, 0.81481, 0.81582,
0.81682, 0.81782, 0.81882, 0.81982, 0.82082, 0.82
182, 0.82282, 0.82382, 0.82482, 0.82583, 0.82683, 0.8278
3, 0.82883, 0.82983, 0.83083, 0.83183, 0.83283, 0.83383,
0.83483, 0.83584, 0.83684, 0.83784, 0.83884, 0.83984,
0.84084, 0.84184, 0.84284, 0.84384, 0.84484, 0.84
585, 0.84685, 0.84785, 0.84885, 0.84985, 0.85085, 0.8518
5, 0.85285, 0.85385, 0.85485, 0.85586, 0.85686, 0.85786,
0.85886, 0.85986, 0.86086, 0.86186, 0.86286, 0.86386,
0.86486, 0.86587, 0.86687, 0.86787, 0.86887, 0.86
987, 0.87087, 0.87187, 0.87287, 0.87387, 0.87487, 0.8758
8, 0.87688, 0.87788, 0.87888, 0.87988, 0.88088, 0.88188,
0.88288, 0.88388, 0.88488, 0.88589, 0.88689, 0.88789,
0.88889, 0.88989, 0.89089, 0.89189, 0.89289, 0.89
389, 0.89489, 0.8959, 0.8969, 0.8979, 0.8989, 0.899
9, 0.9009, 0.9019, 0.9029, 0.9039, 0.9049, 0.90591,
0.90691, 0.90791, 0.90891, 0.90991, 0.91091, 0.91191,
0.91291, 0.91391, 0.91491, 0.91592, 0.91692, 0.91
792, 0.91892, 0.91992, 0.92092, 0.92192, 0.92292, 0.9239
2, 0.92492, 0.92593, 0.92693, 0.92793, 0.92893, 0.92993,
0.93093, 0.93193, 0.93293, 0.93393, 0.93493, 0.93594,
0.93694, 0.93794, 0.93894, 0.93994, 0.94094, 0.94
194, 0.94294, 0.94394, 0.94494, 0.94595, 0.94695, 0.9479
5, 0.94895, 0.94995, 0.95095, 0.95195, 0.95295, 0.95395,
0.95495, 0.95596, 0.95696, 0.95796, 0.95896, 0.95996,
0.96096, 0.96196, 0.96296, 0.96396, 0.96496, 0.96
597, 0.96697, 0.96797, 0.96897, 0.96997, 0.97097, 0.9719
7, 0.97297, 0.97397, 0.97497, 0.97598, 0.97698, 0.97798,
0.97898, 0.97998, 0.98098, 0.98198, 0.98298, 0.98398,
0.98498, 0.98599, 0.98699, 0.98799, 0.98899, 0.98
999, 0.99099, 0.99199, 0.99299, 0.99399, 0.99499, 0.99
6, 0.997, 0.998, 0.999, 1]), array([[1,
1, 1, ..., 0, 0, 0],

```
[ 1, 1, 1, ..., 0.99451, 0.86076,
0]], shape=(2, 1000)), 'Recall', 'Precision'], [array([ 0, 0.001001,
0.002002, 0.003003, 0.004004, 0.005005, 0.006006, 0.007007, 0.
0.008008, 0.009009, 0.01001, 0.011011, 0.012012, 0.013013, 0.01
4014, 0.015015, 0.016016, 0.017017, 0.018018, 0.019019, 0.020
02, 0.021021, 0.022022, 0.023023,
0.024024, 0.025025, 0.026026, 0.027027, 0.028028, 0.029
029, 0.03003, 0.031031, 0.032032, 0.033033, 0.034034, 0.03503
5, 0.036036, 0.037037, 0.038038, 0.039039, 0.04004, 0.041041,
0.042042, 0.043043, 0.044044, 0.045045, 0.046046, 0.047047,
0.048048, 0.049049, 0.05005, 0.051051, 0.052052, 0.053
053, 0.054054, 0.055055, 0.056056, 0.057057, 0.058058, 0.05905
9, 0.06006, 0.061061, 0.062062, 0.063063, 0.064064, 0.065065,
0.066066, 0.067067, 0.068068, 0.069069, 0.07007, 0.071071,
0.072072, 0.073073, 0.074074, 0.075075, 0.076076, 0.077
077, 0.078078, 0.079079, 0.08008, 0.081081, 0.082082, 0.08308
3, 0.084084, 0.085085, 0.086086, 0.087087, 0.088088, 0.089089,
0.09009, 0.091091, 0.092092, 0.093093, 0.094094, 0.095095,
0.096096, 0.097097, 0.098098, 0.099099, 0.1001, 0.1
011, 0.1021, 0.1031, 0.1041, 0.10511, 0.10611, 0.1071
1, 0.10811, 0.10911, 0.11011, 0.11111, 0.11211, 0.11311,
0.11411, 0.11512, 0.11612, 0.11712, 0.11812, 0.11912,
0.12012, 0.12112, 0.12212, 0.12312, 0.12412, 0.12
513, 0.12613, 0.12713, 0.12813, 0.12913, 0.13013, 0.1311
3, 0.13213, 0.13313, 0.13413, 0.13514, 0.13614, 0.13714,
0.13814, 0.13914, 0.14014, 0.14114, 0.14214, 0.14314,
0.14414, 0.14515, 0.14615, 0.14715, 0.14815, 0.14
915, 0.15015, 0.15115, 0.15215, 0.15315, 0.15415, 0.1551
6, 0.15616, 0.15716, 0.15816, 0.15916, 0.16016, 0.16116,
0.16216, 0.16316, 0.16416, 0.16517, 0.16617, 0.16717,
0.16817, 0.16917, 0.17017, 0.17117, 0.17217, 0.17
317, 0.17417, 0.17518, 0.17618, 0.17718, 0.17818, 0.1791
8, 0.18018, 0.18118, 0.18218, 0.18318, 0.18418, 0.18519,
0.18619, 0.18719, 0.18819, 0.18919, 0.19019, 0.19119,
0.19219, 0.19319, 0.19419, 0.1952, 0.1962, 0.1
972, 0.1982, 0.1992, 0.2002, 0.2012, 0.2022, 0.203
2, 0.2042, 0.20521, 0.20621, 0.20721, 0.20821, 0.20921,
0.21021, 0.21121, 0.21221, 0.21321, 0.21421, 0.21522,
0.21622, 0.21722, 0.21822, 0.21922, 0.22022, 0.22
122, 0.22222, 0.22322, 0.22422, 0.22523, 0.22623, 0.2272
3, 0.22823, 0.22923, 0.23023, 0.23123, 0.23223, 0.23323,
0.23423, 0.23524, 0.23624, 0.23724, 0.23824, 0.23924,
0.24024, 0.24124, 0.24224, 0.24324, 0.24424, 0.24
525, 0.24625, 0.24725, 0.24825, 0.24925, 0.25025, 0.2512
5, 0.25225, 0.25325, 0.25425, 0.25526, 0.25626, 0.25726,
0.25826, 0.25926, 0.26026, 0.26126, 0.26226, 0.26326,
0.26426, 0.26527, 0.26627, 0.26727, 0.26827, 0.26
927, 0.27027, 0.27127, 0.27227, 0.27327, 0.27427, 0.2752
8, 0.27628, 0.27728, 0.27828, 0.27928, 0.28028, 0.28128,
0.28228, 0.28328, 0.28428, 0.28529, 0.28629, 0.28729,
0.28829, 0.28929, 0.29029, 0.29129, 0.29229, 0.29
329, 0.29429, 0.2953, 0.2963, 0.2973, 0.2983, 0.299
3, 0.3003, 0.3013, 0.3023, 0.3033, 0.3043, 0.30531,
0.30631, 0.30731, 0.30831, 0.30931, 0.31031, 0.31131,
0.31231, 0.31331, 0.31431, 0.31532, 0.31632, 0.31
732, 0.31832, 0.31932, 0.32032, 0.32132, 0.32232, 0.3233
2, 0.32432, 0.32533, 0.32633, 0.32733, 0.32833, 0.32933,
0.33033, 0.33133, 0.33233, 0.33333, 0.33433, 0.33534,
0.33634, 0.33734, 0.33834, 0.33934, 0.34034, 0.34
```

134, 0.34234, 0.34334, 0.34434, 0.34535, 0.34635, 0.3473
5, 0.34835, 0.34935, 0.35035, 0.35135, 0.35235, 0.35335,
0.35435, 0.35536, 0.35636, 0.35736, 0.35836, 0.35936,
0.36036, 0.36136, 0.36236, 0.36336, 0.36436, 0.36
537, 0.36637, 0.36737, 0.36837, 0.36937, 0.37037, 0.3713
7, 0.37237, 0.37337, 0.37437, 0.37538, 0.37638, 0.37738,
0.37838, 0.37938, 0.38038, 0.38138, 0.38238, 0.38338,
0.38438, 0.38539, 0.38639, 0.38739, 0.38839, 0.38
939, 0.39039, 0.39139, 0.39239, 0.39339, 0.39439, 0.395
4, 0.3964, 0.3974, 0.3984, 0.3994, 0.4004, 0.4014,
0.4024, 0.4034, 0.4044, 0.40541, 0.40641, 0.40741,
0.40841, 0.40941, 0.41041, 0.41141, 0.41241, 0.41
341, 0.41441, 0.41542, 0.41642, 0.41742, 0.41842, 0.4194
2, 0.42042, 0.42142, 0.42242, 0.42342, 0.42442, 0.42543,
0.42643, 0.42743, 0.42843, 0.42943, 0.43043, 0.43143,
0.43243, 0.43343, 0.43443, 0.43544, 0.43644, 0.43
744, 0.43844, 0.43944, 0.44044, 0.44144, 0.44244, 0.4434
4, 0.44444, 0.44545, 0.44645, 0.44745, 0.44845, 0.44945,
0.45045, 0.45145, 0.45245, 0.45345, 0.45445, 0.45546,
0.45646, 0.45746, 0.45846, 0.45946, 0.46046, 0.46
146, 0.46246, 0.46346, 0.46446, 0.46547, 0.46647, 0.4674
7, 0.46847, 0.46947, 0.47047, 0.47147, 0.47247, 0.47347,
0.47447, 0.47548, 0.47648, 0.47748, 0.47848, 0.47948,
0.48048, 0.48148, 0.48248, 0.48348, 0.48448, 0.48
549, 0.48649, 0.48749, 0.48849, 0.48949, 0.49049, 0.4914
9, 0.49249, 0.49349, 0.49449, 0.4955, 0.4965, 0.4975,
0.4985, 0.4995, 0.5005, 0.5015, 0.5025, 0.5035,
0.5045, 0.50551, 0.50651, 0.50751, 0.50851, 0.50
951, 0.51051, 0.51151, 0.51251, 0.51351, 0.51451, 0.5155
2, 0.51652, 0.51752, 0.51852, 0.51952, 0.52052, 0.52152,
0.52252, 0.52352, 0.52452, 0.52553, 0.52653, 0.52753,
0.52853, 0.52953, 0.53053, 0.53153, 0.53253, 0.53
353, 0.53453, 0.53554, 0.53654, 0.53754, 0.53854, 0.5395
4, 0.54054, 0.54154, 0.54254, 0.54354, 0.54454, 0.54555,
0.54655, 0.54755, 0.54855, 0.54955, 0.55055, 0.55155,
0.55255, 0.55355, 0.55455, 0.55556, 0.55656, 0.55
756, 0.55856, 0.55956, 0.56056, 0.56156, 0.56256, 0.5635
6, 0.56456, 0.56557, 0.56657, 0.56757, 0.56857, 0.56957,
0.57057, 0.57157, 0.57257, 0.57357, 0.57457, 0.57558,
0.57658, 0.57758, 0.57858, 0.57958, 0.58058, 0.58
158, 0.58258, 0.58358, 0.58458, 0.58559, 0.58659, 0.5875
9, 0.58859, 0.58959, 0.59059, 0.59159, 0.59259, 0.59359,
0.59459, 0.5956, 0.5966, 0.5976, 0.5986, 0.5996,
0.6006, 0.6016, 0.6026, 0.6036, 0.6046, 0.60
561, 0.60661, 0.60761, 0.60861, 0.60961, 0.61061, 0.6116
1, 0.61261, 0.61361, 0.61461, 0.61562, 0.61662, 0.61762,
0.61862, 0.61962, 0.62062, 0.62162, 0.62262, 0.62362,
0.62462, 0.62563, 0.62663, 0.62763, 0.62863, 0.62
963, 0.63063, 0.63163, 0.63263, 0.63363, 0.63463, 0.6356
4, 0.63664, 0.63764, 0.63864, 0.63964, 0.64064, 0.64164,
0.64264, 0.64364, 0.64464, 0.64565, 0.64665, 0.64765,
0.64865, 0.64965, 0.65065, 0.65165, 0.65265, 0.65
365, 0.65465, 0.65566, 0.65666, 0.65766, 0.65866, 0.6596
6, 0.66066, 0.66166, 0.66266, 0.66366, 0.66466, 0.66567,
0.66667, 0.66767, 0.66867, 0.66967, 0.67067, 0.67167,
0.67267, 0.67367, 0.67467, 0.67568, 0.67668, 0.67
768, 0.67868, 0.67968, 0.68068, 0.68168, 0.68268, 0.6836
8, 0.68468, 0.68569, 0.68669, 0.68769, 0.68869, 0.68969,
0.69069, 0.69169, 0.69269, 0.69369, 0.69469, 0.6957,

0.6967, 0.6977, 0.6987, 0.6997, 0.7007, 0.7
017, 0.7027, 0.7037, 0.7047, 0.70571, 0.70671, 0.7077
1, 0.70871, 0.70971, 0.71071, 0.71171, 0.71271, 0.71371,
0.71471, 0.71572, 0.71672, 0.71772, 0.71872, 0.71972,
0.72072, 0.72172, 0.72272, 0.72372, 0.72472, 0.72
573, 0.72673, 0.72773, 0.72873, 0.72973, 0.73073, 0.7317
3, 0.73273, 0.73373, 0.73473, 0.73574, 0.73674, 0.73774,
0.73874, 0.73974, 0.74074, 0.74174, 0.74274, 0.74374,
0.74474, 0.74575, 0.74675, 0.74775, 0.74875, 0.74
975, 0.75075, 0.75175, 0.75275, 0.75375, 0.75475, 0.7557
6, 0.75676, 0.75776, 0.75876, 0.75976, 0.76076, 0.76176,
0.76276, 0.76376, 0.76476, 0.76577, 0.76677, 0.76777,
0.76877, 0.76977, 0.77077, 0.77177, 0.77277, 0.77
377, 0.77477, 0.77578, 0.77678, 0.77778, 0.77878, 0.7797
8, 0.78078, 0.78178, 0.78278, 0.78378, 0.78478, 0.78579,
0.78679, 0.78779, 0.78879, 0.78979, 0.79079, 0.79179,
0.79279, 0.79379, 0.79479, 0.7958, 0.7968, 0.7
978, 0.7988, 0.7998, 0.8008, 0.8018, 0.8028, 0.803
8, 0.8048, 0.80581, 0.80681, 0.80781, 0.80881, 0.80981,
0.81081, 0.81181, 0.81281, 0.81381, 0.81481, 0.81582,
0.81682, 0.81782, 0.81882, 0.81982, 0.82082, 0.82
182, 0.82282, 0.82382, 0.82482, 0.82583, 0.82683, 0.8278
3, 0.82883, 0.82983, 0.83083, 0.83183, 0.83283, 0.83383,
0.83483, 0.83584, 0.83684, 0.83784, 0.83884, 0.83984,
0.84084, 0.84184, 0.84284, 0.84384, 0.84484, 0.84
585, 0.84685, 0.84785, 0.84885, 0.84985, 0.85085, 0.8518
5, 0.85285, 0.85385, 0.85485, 0.85586, 0.85686, 0.85786,
0.85886, 0.85986, 0.86086, 0.86186, 0.86286, 0.86386,
0.86486, 0.86587, 0.86687, 0.86787, 0.86887, 0.86
987, 0.87087, 0.87187, 0.87287, 0.87387, 0.87487, 0.8758
8, 0.87688, 0.87788, 0.87888, 0.87988, 0.88088, 0.88188,
0.88288, 0.88388, 0.88488, 0.88589, 0.88689, 0.88789,
0.88889, 0.88989, 0.89089, 0.89189, 0.89289, 0.89
389, 0.89489, 0.8959, 0.8969, 0.8979, 0.8989, 0.899
9, 0.9009, 0.9019, 0.9029, 0.9039, 0.9049, 0.90591,
0.90691, 0.90791, 0.90891, 0.90991, 0.91091, 0.91191,
0.91291, 0.91391, 0.91491, 0.91592, 0.91692, 0.91
792, 0.91892, 0.91992, 0.92092, 0.92192, 0.92292, 0.9239
2, 0.92492, 0.92593, 0.92693, 0.92793, 0.92893, 0.92993,
0.93093, 0.93193, 0.93293, 0.93393, 0.93493, 0.93594,
0.93694, 0.93794, 0.93894, 0.93994, 0.94094, 0.94
194, 0.94294, 0.94394, 0.94494, 0.94595, 0.94695, 0.9479
5, 0.94895, 0.94995, 0.95095, 0.95195, 0.95295, 0.95395,
0.95495, 0.95596, 0.95696, 0.95796, 0.95896, 0.95996,
0.96096, 0.96196, 0.96296, 0.96396, 0.96496, 0.96
597, 0.96697, 0.96797, 0.96897, 0.96997, 0.97097, 0.9719
7, 0.97297, 0.97397, 0.97497, 0.97598, 0.97698, 0.97798,
0.97898, 0.97998, 0.98098, 0.98198, 0.98298, 0.98398,
0.98498, 0.98599, 0.98699, 0.98799, 0.98899, 0.98
999, 0.99099, 0.99199, 0.99299, 0.99399, 0.99499, 0.99
6, 0.997, 0.998, 0.999, 1]), array([[0.42639,
0.42639, 0.6941, ..., 0, 0, 0],
[0.59584, 0.59584, 0.75006, ..., 0, 0,
0]], shape=(2, 1000)), 'Confidence', 'F1'], [array([
0.002002, 0.003003, 0.004004, 0.005005, 0.006006, 0.007007, 0.
008008, 0.009009, 0.01001, 0.011011, 0.012012, 0.013013, 0.01
4014, 0.015015, 0.016016, 0.017017, 0.018018, 0.019019, 0.020
02, 0.021021, 0.022022, 0.023023,
0.024024, 0.025025, 0.026026, 0.027027, 0.028028, 0.029

029, 0.03003, 0.031031, 0.032032, 0.033033, 0.034034, 0.03503
5, 0.036036, 0.037037, 0.038038, 0.039039, 0.04004, 0.041041,
0.042042, 0.043043, 0.044044, 0.045045, 0.046046, 0.047047,
0.048048, 0.049049, 0.05005, 0.051051, 0.052052, 0.053
053, 0.054054, 0.055055, 0.056056, 0.057057, 0.058058, 0.05905
9, 0.06006, 0.061061, 0.062062, 0.063063, 0.064064, 0.065065,
0.066066, 0.067067, 0.068068, 0.069069, 0.07007, 0.071071,
0.072072, 0.073073, 0.074074, 0.075075, 0.076076, 0.077
077, 0.078078, 0.079079, 0.08008, 0.081081, 0.082082, 0.08308
3, 0.084084, 0.085085, 0.086086, 0.087087, 0.088088, 0.089089,
0.09009, 0.091091, 0.092092, 0.093093, 0.094094, 0.095095,
0.096096, 0.097097, 0.098098, 0.099099, 0.1001, 0.1
011, 0.1021, 0.1031, 0.1041, 0.10511, 0.10611, 0.1071
1, 0.10811, 0.10911, 0.11011, 0.11111, 0.11211, 0.11311,
0.11411, 0.11512, 0.11612, 0.11712, 0.11812, 0.11912,
0.12012, 0.12112, 0.12212, 0.12312, 0.12412, 0.12
513, 0.12613, 0.12713, 0.12813, 0.12913, 0.13013, 0.1311
3, 0.13213, 0.13313, 0.13413, 0.13514, 0.13614, 0.13714,
0.13814, 0.13914, 0.14014, 0.14114, 0.14214, 0.14314,
0.14414, 0.14515, 0.14615, 0.14715, 0.14815, 0.14
915, 0.15015, 0.15115, 0.15215, 0.15315, 0.15415, 0.1551
6, 0.15616, 0.15716, 0.15816, 0.15916, 0.16016, 0.16116,
0.16216, 0.16316, 0.16416, 0.16517, 0.16617, 0.16717,
0.16817, 0.16917, 0.17017, 0.17117, 0.17217, 0.17
317, 0.17417, 0.17518, 0.17618, 0.17718, 0.17818, 0.1791
8, 0.18018, 0.18118, 0.18218, 0.18318, 0.18418, 0.18519,
0.18619, 0.18719, 0.18819, 0.18919, 0.19019, 0.19119,
0.19219, 0.19319, 0.19419, 0.1952, 0.1962, 0.1
972, 0.1982, 0.1992, 0.2002, 0.2012, 0.2022, 0.203
2, 0.2042, 0.20521, 0.20621, 0.20721, 0.20821, 0.20921,
0.21021, 0.21121, 0.21221, 0.21321, 0.21421, 0.21522,
0.21622, 0.21722, 0.21822, 0.21922, 0.22022, 0.22
122, 0.22222, 0.22322, 0.22422, 0.22523, 0.22623, 0.2272
3, 0.22823, 0.22923, 0.23023, 0.23123, 0.23223, 0.23323,
0.23423, 0.23524, 0.23624, 0.23724, 0.23824, 0.23924,
0.24024, 0.24124, 0.24224, 0.24324, 0.24424, 0.24
525, 0.24625, 0.24725, 0.24825, 0.24925, 0.25025, 0.2512
5, 0.25225, 0.25325, 0.25425, 0.25526, 0.25626, 0.25726,
0.25826, 0.25926, 0.26026, 0.26126, 0.26226, 0.26326,
0.26426, 0.26527, 0.26627, 0.26727, 0.26827, 0.26
927, 0.27027, 0.27127, 0.27227, 0.27327, 0.27427, 0.2752
8, 0.27628, 0.27728, 0.27828, 0.27928, 0.28028, 0.28128,
0.28228, 0.28328, 0.28428, 0.28529, 0.28629, 0.28729,
0.28829, 0.28929, 0.29029, 0.29129, 0.29229, 0.29
329, 0.29429, 0.2953, 0.2963, 0.2973, 0.2983, 0.299
3, 0.3003, 0.3013, 0.3023, 0.3033, 0.3043, 0.30531,
0.30631, 0.30731, 0.30831, 0.30931, 0.31031, 0.31131,
0.31231, 0.31331, 0.31431, 0.31532, 0.31632, 0.31
732, 0.31832, 0.31932, 0.32032, 0.32132, 0.32232, 0.3233
2, 0.32432, 0.32533, 0.32633, 0.32733, 0.32833, 0.32933,
0.33033, 0.33133, 0.33233, 0.33333, 0.33433, 0.33534,
0.33634, 0.33734, 0.33834, 0.33934, 0.34034, 0.34
134, 0.34234, 0.34334, 0.34434, 0.34535, 0.34635, 0.3473
5, 0.34835, 0.34935, 0.35035, 0.35135, 0.35235, 0.35335,
0.35435, 0.35536, 0.35636, 0.35736, 0.35836, 0.35936,
0.36036, 0.36136, 0.36236, 0.36336, 0.36436, 0.36
537, 0.36637, 0.36737, 0.36837, 0.36937, 0.37037, 0.3713
7, 0.37237, 0.37337, 0.37437, 0.37538, 0.37638, 0.37738,
0.37838, 0.37938, 0.38038, 0.38138, 0.38238, 0.38338,

	0.38438,	0.38539,	0.38639,	0.38739,	0.38839,	0.38
939,	0.39039,	0.39139,	0.39239,	0.39339,	0.39439,	0.395
4,	0.3964,	0.3974,	0.3984,	0.3994,	0.4004,	0.4014,
0.4024,	0.4034,	0.4044,	0.40541,	0.40641,	0.40741,	
	0.40841,	0.40941,	0.41041,	0.41141,	0.41241,	0.41
341,	0.41441,	0.41542,	0.41642,	0.41742,	0.41842,	0.4194
2,	0.42042,	0.42142,	0.42242,	0.42342,	0.42442,	0.42543,
0.42643,	0.42743,	0.42843,	0.42943,	0.43043,	0.43143,	
	0.43243,	0.43343,	0.43443,	0.43544,	0.43644,	0.43
744,	0.43844,	0.43944,	0.44044,	0.44144,	0.44244,	0.4434
4,	0.44444,	0.44545,	0.44645,	0.44745,	0.44845,	0.44945,
0.45045,	0.45145,	0.45245,	0.45345,	0.45445,	0.45546,	
	0.45646,	0.45746,	0.45846,	0.45946,	0.46046,	0.46
146,	0.46246,	0.46346,	0.46446,	0.46547,	0.46647,	0.4674
7,	0.46847,	0.46947,	0.47047,	0.47147,	0.47247,	0.47347,
0.47447,	0.47548,	0.47648,	0.47748,	0.47848,	0.47948,	
	0.48048,	0.48148,	0.48248,	0.48348,	0.48448,	0.48
549,	0.48649,	0.48749,	0.48849,	0.48949,	0.49049,	0.4914
9,	0.49249,	0.49349,	0.49449,	0.4955,	0.4965,	0.4975,
0.4985,	0.4995,	0.5005,	0.5015,	0.5025,	0.5035,	
	0.5045,	0.50551,	0.50651,	0.50751,	0.50851,	0.50
951,	0.51051,	0.51151,	0.51251,	0.51351,	0.51451,	0.5155
2,	0.51652,	0.51752,	0.51852,	0.51952,	0.52052,	0.52152,
0.52252,	0.52352,	0.52452,	0.52553,	0.52653,	0.52753,	
	0.52853,	0.52953,	0.53053,	0.53153,	0.53253,	0.53
353,	0.53453,	0.53554,	0.53654,	0.53754,	0.53854,	0.5395
4,	0.54054,	0.54154,	0.54254,	0.54354,	0.54454,	0.54555,
0.54655,	0.54755,	0.54855,	0.54955,	0.55055,	0.55155,	
	0.55255,	0.55355,	0.55455,	0.55556,	0.55656,	0.55
756,	0.55856,	0.55956,	0.56056,	0.56156,	0.56256,	0.5635
6,	0.56456,	0.56557,	0.56657,	0.56757,	0.56857,	0.56957,
0.57057,	0.57157,	0.57257,	0.57357,	0.57457,	0.57558,	
	0.57658,	0.57758,	0.57858,	0.57958,	0.58058,	0.58
158,	0.58258,	0.58358,	0.58458,	0.58559,	0.58659,	0.5875
9,	0.58859,	0.58959,	0.59059,	0.59159,	0.59259,	0.59359,
0.59459,	0.5956,	0.5966,	0.5976,	0.5986,	0.5996,	
	0.6006,	0.6016,	0.6026,	0.6036,	0.6046,	0.60
561,	0.60661,	0.60761,	0.60861,	0.60961,	0.61061,	0.6116
1,	0.61261,	0.61361,	0.61461,	0.61562,	0.61662,	0.61762,
0.61862,	0.61962,	0.62062,	0.62162,	0.62262,	0.62362,	
	0.62462,	0.62563,	0.62663,	0.62763,	0.62863,	0.62
963,	0.63063,	0.63163,	0.63263,	0.63363,	0.63463,	0.6356
4,	0.63664,	0.63764,	0.63864,	0.63964,	0.64064,	0.64164,
0.64264,	0.64364,	0.64464,	0.64565,	0.64665,	0.64765,	
	0.64865,	0.64965,	0.65065,	0.65165,	0.65265,	0.65
365,	0.65465,	0.65566,	0.65666,	0.65766,	0.65866,	0.6596
6,	0.66066,	0.66166,	0.66266,	0.66366,	0.66466,	0.66567,
0.66667,	0.66767,	0.66867,	0.66967,	0.67067,	0.67167,	
	0.67267,	0.67367,	0.67467,	0.67568,	0.67668,	0.67
768,	0.67868,	0.67968,	0.68068,	0.68168,	0.68268,	0.6836
8,	0.68468,	0.68569,	0.68669,	0.68769,	0.68869,	0.68969,
0.69069,	0.69169,	0.69269,	0.69369,	0.69469,	0.6957,	
	0.6967,	0.6977,	0.6987,	0.6997,	0.7007,	0.7
017,	0.7027,	0.7037,	0.7047,	0.70571,	0.70671,	0.7077
1,	0.70871,	0.70971,	0.71071,	0.71171,	0.71271,	0.71371,
0.71471,	0.71572,	0.71672,	0.71772,	0.71872,	0.71972,	
	0.72072,	0.72172,	0.72272,	0.72372,	0.72472,	0.72
573,	0.72673,	0.72773,	0.72873,	0.72973,	0.73073,	0.7317
3,	0.73273,	0.73373,	0.73473,	0.73574,	0.73674,	0.73774,

0.73874, 0.73974, 0.74074, 0.74174, 0.74274, 0.74374,
0.74474, 0.74575, 0.74675, 0.74775, 0.74875, 0.74
975, 0.75075, 0.75175, 0.75275, 0.75375, 0.75475, 0.7557
6, 0.75676, 0.75776, 0.75876, 0.75976, 0.76076, 0.76176,
0.76276, 0.76376, 0.76476, 0.76577, 0.76677, 0.76777,
0.76877, 0.76977, 0.77077, 0.77177, 0.77277, 0.77
377, 0.77477, 0.77578, 0.77678, 0.77778, 0.77878, 0.7797
8, 0.78078, 0.78178, 0.78278, 0.78378, 0.78478, 0.78579,
0.78679, 0.78779, 0.78879, 0.78979, 0.79079, 0.79179,
0.79279, 0.79379, 0.79479, 0.7958, 0.7968, 0.7
978, 0.7988, 0.7998, 0.8008, 0.8018, 0.8028, 0.803
8, 0.8048, 0.80581, 0.80681, 0.80781, 0.80881, 0.80981,
0.81081, 0.81181, 0.81281, 0.81381, 0.81481, 0.81582,
0.81682, 0.81782, 0.81882, 0.81982, 0.82082, 0.82
182, 0.82282, 0.82382, 0.82482, 0.82583, 0.82683, 0.8278
3, 0.82883, 0.82983, 0.83083, 0.83183, 0.83283, 0.83383,
0.83483, 0.83584, 0.83684, 0.83784, 0.83884, 0.83984,
0.84084, 0.84184, 0.84284, 0.84384, 0.84484, 0.84
585, 0.84685, 0.84785, 0.84885, 0.84985, 0.85085, 0.8518
5, 0.85285, 0.85385, 0.85485, 0.85586, 0.85686, 0.85786,
0.85886, 0.85986, 0.86086, 0.86186, 0.86286, 0.86386,
0.86486, 0.86587, 0.86687, 0.86787, 0.86887, 0.86
987, 0.87087, 0.87187, 0.87287, 0.87387, 0.87487, 0.8758
8, 0.87688, 0.87788, 0.87888, 0.87988, 0.88088, 0.88188,
0.88288, 0.88388, 0.88488, 0.88589, 0.88689, 0.88789,
0.88889, 0.88989, 0.89089, 0.89189, 0.89289, 0.89
389, 0.89489, 0.8959, 0.8969, 0.8979, 0.8989, 0.899
9, 0.9009, 0.9019, 0.9029, 0.9039, 0.9049, 0.90591,
0.90691, 0.90791, 0.90891, 0.90991, 0.91091, 0.91191,
0.91291, 0.91391, 0.91491, 0.91592, 0.91692, 0.91
792, 0.91892, 0.91992, 0.92092, 0.92192, 0.92292, 0.9239
2, 0.92492, 0.92593, 0.92693, 0.92793, 0.92893, 0.92993,
0.93093, 0.93193, 0.93293, 0.93393, 0.93493, 0.93594,
0.93694, 0.93794, 0.93894, 0.93994, 0.94094, 0.94
194, 0.94294, 0.94394, 0.94494, 0.94595, 0.94695, 0.9479
5, 0.94895, 0.94995, 0.95095, 0.95195, 0.95295, 0.95395,
0.95495, 0.95596, 0.95696, 0.95796, 0.95896, 0.95996,
0.96096, 0.96196, 0.96296, 0.96396, 0.96496, 0.96
597, 0.96697, 0.96797, 0.96897, 0.96997, 0.97097, 0.9719
7, 0.97297, 0.97397, 0.97497, 0.97598, 0.97698, 0.97798,
0.97898, 0.97998, 0.98098, 0.98198, 0.98298, 0.98398,
0.98498, 0.98599, 0.98699, 0.98799, 0.98899, 0.98
999, 0.99099, 0.99199, 0.99299, 0.99399, 0.99499, 0.99
6, 0.997, 0.998, 0.999, 1]), array([[0.27169,
0.27169, 0.53433, ..., 1, 1, 1],
[0.42434, 0.42434, 0.60008, ..., 1, 1,
1]], shape=(2, 1000)), 'Confidence', 'Precision'], [array([0, 0.001
001, 0.002002, 0.003003, 0.004004, 0.005005, 0.006006, 0.00700
7, 0.008008, 0.009009, 0.01001, 0.011011, 0.012012, 0.013013,
0.014014, 0.015015, 0.016016, 0.017017, 0.018018, 0.019019,
0.02002, 0.021021, 0.022022, 0.023023,
0.024024, 0.025025, 0.026026, 0.027027, 0.028028, 0.029
029, 0.03003, 0.031031, 0.032032, 0.033033, 0.034034, 0.03503
5, 0.036036, 0.037037, 0.038038, 0.039039, 0.04004, 0.041041,
0.042042, 0.043043, 0.044044, 0.045045, 0.046046, 0.047047,
0.048048, 0.049049, 0.05005, 0.051051, 0.052052, 0.053
053, 0.054054, 0.055055, 0.056056, 0.057057, 0.058058, 0.05905
9, 0.06006, 0.061061, 0.062062, 0.063063, 0.064064, 0.065065,
0.066066, 0.067067, 0.068068, 0.069069, 0.07007, 0.071071,

	0.072072,	0.073073,	0.074074,	0.075075,	0.076076,	0.077
077,	0.078078,	0.079079,	0.08008,	0.081081,	0.082082,	0.08308
3,	0.084084,	0.085085,	0.086086,	0.087087,	0.088088,	0.089089,
0.09009,	0.091091,	0.092092,	0.093093,	0.094094,	0.095095,	
	0.096096,	0.097097,	0.098098,	0.099099,	0.1001,	0.1
011,	0.1021,	0.1031,	0.1041,	0.10511,	0.10611,	0.1071
1,	0.10811,	0.10911,	0.11011,	0.11111,	0.11211,	0.11311,
0.11411,	0.11512,	0.11612,	0.11712,	0.11812,	0.11912,	
	0.12012,	0.12112,	0.12212,	0.12312,	0.12412,	0.12
513,	0.12613,	0.12713,	0.12813,	0.12913,	0.13013,	0.1311
3,	0.13213,	0.13313,	0.13413,	0.13514,	0.13614,	0.13714,
0.13814,	0.13914,	0.14014,	0.14114,	0.14214,	0.14314,	
	0.14414,	0.14515,	0.14615,	0.14715,	0.14815,	0.14
915,	0.15015,	0.15115,	0.15215,	0.15315,	0.15415,	0.1551
6,	0.15616,	0.15716,	0.15816,	0.15916,	0.16016,	0.16116,
0.16216,	0.16316,	0.16416,	0.16517,	0.16617,	0.16717,	
	0.16817,	0.16917,	0.17017,	0.17117,	0.17217,	0.17
317,	0.17417,	0.17518,	0.17618,	0.17718,	0.17818,	0.1791
8,	0.18018,	0.18118,	0.18218,	0.18318,	0.18418,	0.18519,
0.18619,	0.18719,	0.18819,	0.18919,	0.19019,	0.19119,	
	0.19219,	0.19319,	0.19419,	0.1952,	0.1962,	0.1
972,	0.1982,	0.1992,	0.2002,	0.2012,	0.2022,	0.203
2,	0.2042,	0.20521,	0.20621,	0.20721,	0.20821,	0.20921,
0.21021,	0.21121,	0.21221,	0.21321,	0.21421,	0.21522,	
	0.21622,	0.21722,	0.21822,	0.21922,	0.22022,	0.22
122,	0.22222,	0.22322,	0.22422,	0.22523,	0.22623,	0.2272
3,	0.22823,	0.22923,	0.23023,	0.23123,	0.23223,	0.23323,
0.23423,	0.23524,	0.23624,	0.23724,	0.23824,	0.23924,	
	0.24024,	0.24124,	0.24224,	0.24324,	0.24424,	0.24
525,	0.24625,	0.24725,	0.24825,	0.24925,	0.25025,	0.2512
5,	0.25225,	0.25325,	0.25425,	0.25526,	0.25626,	0.25726,
0.25826,	0.25926,	0.26026,	0.26126,	0.26226,	0.26326,	
	0.26426,	0.26527,	0.26627,	0.26727,	0.26827,	0.26
927,	0.27027,	0.27127,	0.27227,	0.27327,	0.27427,	0.2752
8,	0.27628,	0.27728,	0.27828,	0.27928,	0.28028,	0.28128,
0.28228,	0.28328,	0.28428,	0.28529,	0.28629,	0.28729,	
	0.28829,	0.28929,	0.29029,	0.29129,	0.29229,	0.29
329,	0.29429,	0.2953,	0.2963,	0.2973,	0.2983,	0.299
3,	0.3003,	0.3013,	0.3023,	0.3033,	0.3043,	0.30531,
0.30631,	0.30731,	0.30831,	0.30931,	0.31031,	0.31131,	
	0.31231,	0.31331,	0.31431,	0.31532,	0.31632,	0.31
732,	0.31832,	0.31932,	0.32032,	0.32132,	0.32232,	0.3233
2,	0.32432,	0.32533,	0.32633,	0.32733,	0.32833,	0.32933,
0.33033,	0.33133,	0.33233,	0.33333,	0.33433,	0.33534,	
	0.33634,	0.33734,	0.33834,	0.33934,	0.34034,	0.34
134,	0.34234,	0.34334,	0.34434,	0.34535,	0.34635,	0.3473
5,	0.34835,	0.34935,	0.35035,	0.35135,	0.35235,	0.35335,
0.35435,	0.35536,	0.35636,	0.35736,	0.35836,	0.35936,	
	0.36036,	0.36136,	0.36236,	0.36336,	0.36436,	0.36
537,	0.36637,	0.36737,	0.36837,	0.36937,	0.37037,	0.3713
7,	0.37237,	0.37337,	0.37437,	0.37538,	0.37638,	0.37738,
0.37838,	0.37938,	0.38038,	0.38138,	0.38238,	0.38338,	
	0.38438,	0.38539,	0.38639,	0.38739,	0.38839,	0.38
939,	0.39039,	0.39139,	0.39239,	0.39339,	0.39439,	0.395
4,	0.3964,	0.3974,	0.3984,	0.3994,	0.4004,	0.4014,
0.4024,	0.4034,	0.4044,	0.40541,	0.40641,	0.40741,	
	0.40841,	0.40941,	0.41041,	0.41141,	0.41241,	0.41
341,	0.41441,	0.41542,	0.41642,	0.41742,	0.41842,	0.4194
2,	0.42042,	0.42142,	0.42242,	0.42342,	0.42442,	0.42543,

0.42643, 0.42743, 0.42843, 0.42943, 0.43043, 0.43143,
0.43243, 0.43343, 0.43443, 0.43544, 0.43644, 0.43
744, 0.43844, 0.43944, 0.44044, 0.44144, 0.44244, 0.4434
4, 0.44444, 0.44545, 0.44645, 0.44745, 0.44845, 0.44945,
0.45045, 0.45145, 0.45245, 0.45345, 0.45445, 0.45546,
0.45646, 0.45746, 0.45846, 0.45946, 0.46046, 0.46
146, 0.46246, 0.46346, 0.46446, 0.46547, 0.46647, 0.4674
7, 0.46847, 0.46947, 0.47047, 0.47147, 0.47247, 0.47347,
0.47447, 0.47548, 0.47648, 0.47748, 0.47848, 0.47948,
0.48048, 0.48148, 0.48248, 0.48348, 0.48448, 0.48
549, 0.48649, 0.48749, 0.48849, 0.48949, 0.49049, 0.4914
9, 0.49249, 0.49349, 0.49449, 0.4955, 0.4965, 0.4975,
0.4985, 0.4995, 0.5005, 0.5015, 0.5025, 0.5035,
0.5045, 0.50551, 0.50651, 0.50751, 0.50851, 0.50
951, 0.51051, 0.51151, 0.51251, 0.51351, 0.51451, 0.5155
2, 0.51652, 0.51752, 0.51852, 0.51952, 0.52052, 0.52152,
0.52252, 0.52352, 0.52452, 0.52553, 0.52653, 0.52753,
0.52853, 0.52953, 0.53053, 0.53153, 0.53253, 0.53
353, 0.53453, 0.53554, 0.53654, 0.53754, 0.53854, 0.5395
4, 0.54054, 0.54154, 0.54254, 0.54354, 0.54454, 0.54555,
0.54655, 0.54755, 0.54855, 0.54955, 0.55055, 0.55155,
0.55255, 0.55355, 0.55455, 0.55556, 0.55656, 0.55
756, 0.55856, 0.55956, 0.56056, 0.56156, 0.56256, 0.5635
6, 0.56456, 0.56557, 0.56657, 0.56757, 0.56857, 0.56957,
0.57057, 0.57157, 0.57257, 0.57357, 0.57457, 0.57558,
0.57658, 0.57758, 0.57858, 0.57958, 0.58058, 0.58
158, 0.58258, 0.58358, 0.58458, 0.58559, 0.58659, 0.5875
9, 0.58859, 0.58959, 0.59059, 0.59159, 0.59259, 0.59359,
0.59459, 0.5956, 0.5966, 0.5976, 0.5986, 0.5996,
0.6006, 0.6016, 0.6026, 0.6036, 0.6046, 0.60
561, 0.60661, 0.60761, 0.60861, 0.60961, 0.61061, 0.6116
1, 0.61261, 0.61361, 0.61461, 0.61562, 0.61662, 0.61762,
0.61862, 0.61962, 0.62062, 0.62162, 0.62262, 0.62362,
0.62462, 0.62563, 0.62663, 0.62763, 0.62863, 0.62
963, 0.63063, 0.63163, 0.63263, 0.63363, 0.63463, 0.6356
4, 0.63664, 0.63764, 0.63864, 0.63964, 0.64064, 0.64164,
0.64264, 0.64364, 0.64464, 0.64565, 0.64665, 0.64765,
0.64865, 0.64965, 0.65065, 0.65165, 0.65265, 0.65
365, 0.65465, 0.65566, 0.65666, 0.65766, 0.65866, 0.6596
6, 0.66066, 0.66166, 0.66266, 0.66366, 0.66466, 0.66567,
0.66667, 0.66767, 0.66867, 0.66967, 0.67067, 0.67167,
0.67267, 0.67367, 0.67467, 0.67568, 0.67668, 0.67
768, 0.67868, 0.67968, 0.68068, 0.68168, 0.68268, 0.6836
8, 0.68468, 0.68569, 0.68669, 0.68769, 0.68869, 0.68969,
0.69069, 0.69169, 0.69269, 0.69369, 0.69469, 0.6957,
0.6967, 0.6977, 0.6987, 0.6997, 0.7007, 0.7
017, 0.7027, 0.7037, 0.7047, 0.70571, 0.70671, 0.7077
1, 0.70871, 0.70971, 0.71071, 0.71171, 0.71271, 0.71371,
0.71471, 0.71572, 0.71672, 0.71772, 0.71872, 0.71972,
0.72072, 0.72172, 0.72272, 0.72372, 0.72472, 0.72
573, 0.72673, 0.72773, 0.72873, 0.72973, 0.73073, 0.7317
3, 0.73273, 0.73373, 0.73473, 0.73574, 0.73674, 0.73774,
0.73874, 0.73974, 0.74074, 0.74174, 0.74274, 0.74374,
0.74474, 0.74575, 0.74675, 0.74775, 0.74875, 0.74
975, 0.75075, 0.75175, 0.75275, 0.75375, 0.75475, 0.7557
6, 0.75676, 0.75776, 0.75876, 0.75976, 0.76076, 0.76176,
0.76276, 0.76376, 0.76476, 0.76577, 0.76677, 0.76777,
0.76877, 0.76977, 0.77077, 0.77177, 0.77277, 0.77
377, 0.77477, 0.77578, 0.77678, 0.77778, 0.77878, 0.7797

```

8,      0.78078,      0.78178,      0.78278,      0.78378,      0.78478,      0.78579,
0.78679,      0.78779,      0.78879,      0.78979,      0.79079,      0.79179,
      0.79279,      0.79379,      0.79479,      0.7958,      0.7968,      0.7
978,      0.7988,      0.7998,      0.8008,      0.8018,      0.8028,      0.803
8,      0.8048,      0.80581,      0.80681,      0.80781,      0.80881,      0.80981,
0.81081,      0.81181,      0.81281,      0.81381,      0.81481,      0.81582,
      0.81682,      0.81782,      0.81882,      0.81982,      0.82082,      0.82
182,      0.82282,      0.82382,      0.82482,      0.82583,      0.82683,      0.8278
3,      0.82883,      0.82983,      0.83083,      0.83183,      0.83283,      0.83383,
0.83483,      0.83584,      0.83684,      0.83784,      0.83884,      0.83984,
      0.84084,      0.84184,      0.84284,      0.84384,      0.84484,      0.84
585,      0.84685,      0.84785,      0.84885,      0.84985,      0.85085,      0.8518
5,      0.85285,      0.85385,      0.85485,      0.85586,      0.85686,      0.85786,
0.85886,      0.85986,      0.86086,      0.86186,      0.86286,      0.86386,
      0.86486,      0.86587,      0.86687,      0.86787,      0.86887,      0.86
987,      0.87087,      0.87187,      0.87287,      0.87387,      0.87487,      0.8758
8,      0.87688,      0.87788,      0.87888,      0.87988,      0.88088,      0.88188,
0.88288,      0.88388,      0.88488,      0.88589,      0.88689,      0.88789,
      0.88889,      0.88989,      0.89089,      0.89189,      0.89289,      0.89
389,      0.89489,      0.8959,      0.8969,      0.8979,      0.8989,      0.899
9,      0.9009,      0.9019,      0.9029,      0.9039,      0.9049,      0.90591,
0.90691,      0.90791,      0.90891,      0.90991,      0.91091,      0.91191,
      0.91291,      0.91391,      0.91491,      0.91592,      0.91692,      0.91
792,      0.91892,      0.91992,      0.92092,      0.92192,      0.92292,      0.9239
2,      0.92492,      0.92593,      0.92693,      0.92793,      0.92893,      0.92993,
0.93093,      0.93193,      0.93293,      0.93393,      0.93493,      0.93594,
      0.93694,      0.93794,      0.93894,      0.93994,      0.94094,      0.94
194,      0.94294,      0.94394,      0.94494,      0.94595,      0.94695,      0.9479
5,      0.94895,      0.94995,      0.95095,      0.95195,      0.95295,      0.95395,
0.95495,      0.95596,      0.95696,      0.95796,      0.95896,      0.95996,
      0.96096,      0.96196,      0.96296,      0.96396,      0.96496,      0.96
597,      0.96697,      0.96797,      0.96897,      0.96997,      0.97097,      0.9719
7,      0.97297,      0.97397,      0.97497,      0.97598,      0.97698,      0.97798,
0.97898,      0.97998,      0.98098,      0.98198,      0.98298,      0.98398,
      0.98498,      0.98599,      0.98699,      0.98799,      0.98899,      0.98
999,      0.99099,      0.99199,      0.99299,      0.99399,      0.99499,      0.99
6,      0.997,      0.998,      0.999,      1]], array([[      0.9902,
0.9902,      0.9902, ...,      0,      0,      0],
[      1,      1,      1, ...,      0,      0,
0]], shape=(2, 1000)), 'Confidence', 'Recall']]
fitness: 0.642312256142742
keys: ['metrics/precision(B)', 'metrics/recall(B)', 'metrics/mAP50(B)', 'metric
s/mAP50-95(B)']
maps: array([      0.71174,      0.57288])
names: {0: 'shoes', 1: 'stamp'}
nt_per_class: array([408, 544])
nt_per_image: array([204, 273])
results_dict: {'metrics/precision(B)': 0.941407568533371, 'metrics/recall(B)':
0.9708940980387059, 'metrics/mAP50(B)': 0.9835965513659438, 'metrics/mAP50-95
(B)': 0.642312256142742, 'fitness': 0.642312256142742}
save_dir: WindowsPath('C:/Users/LOQ5060/Desktop/CV-YOLO/runs/stamp/stamp_detec
t')
speed: {'preprocess': 0.2509366300668322, 'inference': 1.611039926781534, 'los
s': 0.00037728915854114966, 'postprocess': 1.4234029306499663}
stats: {'tp': [], 'conf': [], 'pred_cls': [], 'target_cls': [], 'target_img':
[]}]

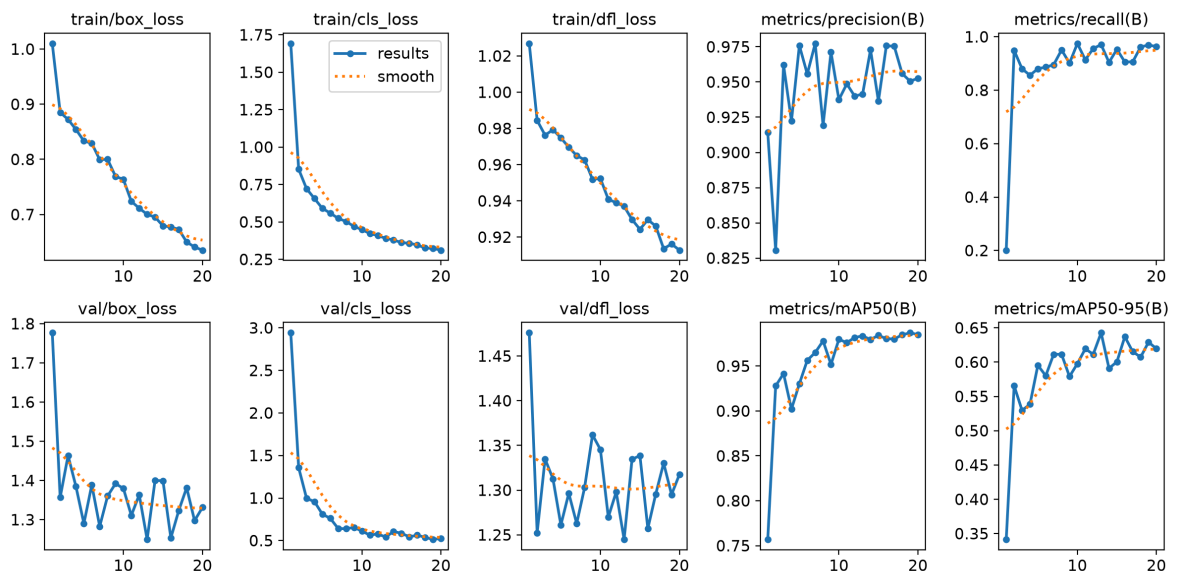
```

 **설명** — 사전학습 가중치에서 시작해 stamp 데이터로 파인튜닝한다.

인자	의미
<code>data</code>	데이터 설정 파일 (경로 + 클래스 이름)
<code>epochs=20</code>	학습 데이터를 20번 반복
<code>imgsz=640</code>	입력 크기. 원본(416)을 640으로 키워 넣는다
<code>batch=16</code>	한 번에 16장씩
<code>device=0</code>	첫 번째 GPU
<code>project/exist_ok/workers/seed</code>	(추가) 저장 위치 고정, 덮어쓰기 허용, Windows용 로더 수, 재현성

로그의 `box_loss` (위치 오차), `cls_loss` (분류 오차), `dfl_loss` (박스 경계 분포 오차)가 에폭마다 줄어드는지 본다.

```
In [50]: display(Image.open(RUNS / "stamp_detect" / "results.png"))
```



학습 곡선 해석

- 학습 손실(`train/box_loss` , `cls_loss` , `dfl_loss`)은 20 에폭 동안 **꾸준히 감소** 했다.
- 검증 손실(`val/box_loss` , `val/dfl_loss`)은 5~10 에폭 이후 더 줄지 않고 오르 내린다.
- mAP50은 10 에폭쯤 **0.98에서 포화**됐고, mAP50-95는 0.58~0.64 사이에서 흔들리다 **13 에폭에서 최고(0.642)** 였다. Ultralytics는 이 최고 시점을 `best.pt` 로 저장한다.

→ 20 에폭이면 충분히 수렴했다. 더 학습해도 크게 좋아지지 않을 것으로 보이며, 이 예상은 개인 실험 2-1(50 에폭)에서 확인한다.

```
In [51]: metrics = model.val()
print("🔍 평가 결과:", metrics)
```

Ultralytics 8.4.154 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU, 8151MiB)

Model summary (fused): 72 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs

val: Fast image access (ping: 0.10.0 ms, read: 92.421.6 MB/s, size: 27.1 KB)

val: Scanning C:\Users\LOQ5060\Desktop\CV-YOLO\data\stamp\valid\labels.cache... 273 images, 0 backgrounds, 0 corrupt: 100% ————— 273/273 0.0s

	all	273	952	0.942	0.971	0.984
0.642						
	shoes	204	408	0.944	0.944	0.972
0.71						
	stamp	273	544	0.941	0.998	0.995
0.574						

Speed: 1.3ms preprocess, 2.9ms inference, 0.0ms loss, 1.9ms postprocess per image

Results saved to C:\Users\LOQ5060\runs\detect\val

 평가 결과: ultralytics.utils.metrics.DetMetrics object with attributes:

ap_class_index: array([0, 1])

box: ultralytics.utils.metrics.Metric object

... (166줄 생략) ...

```
0.91291, 0.91391, 0.91491, 0.91592, 0.91692, 0.9179
2, 0.91892, 0.91992, 0.92092, 0.92192, 0.92292, 0.92392,
0.92492, 0.92593, 0.92693, 0.92793, 0.92893, 0.92993, 0.93
093, 0.93193, 0.93293, 0.93393, 0.93493, 0.93594,
0.93694, 0.93794, 0.93894, 0.93994, 0.94094, 0.9419
4, 0.94294, 0.94394, 0.94494, 0.94595, 0.94695, 0.94795,
0.94895, 0.94995, 0.95095, 0.95195, 0.95295, 0.95395, 0.95
495, 0.95596, 0.95696, 0.95796, 0.95896, 0.95996,
0.96096, 0.96196, 0.96296, 0.96396, 0.96496, 0.9659
7, 0.96697, 0.96797, 0.96897, 0.96997, 0.97097, 0.97197,
0.97297, 0.97397, 0.97497, 0.97598, 0.97698, 0.97798, 0.97
898, 0.97998, 0.98098, 0.98198, 0.98298, 0.98398,
0.98498, 0.98599, 0.98699, 0.98799, 0.98899, 0.9899
9, 0.99099, 0.99199, 0.99299, 0.99399, 0.99499, 0.996,
0.997, 0.998, 0.999, 1]), array([[ 0.9902, 0.9902,
0.9902, ..., 0, 0, 0, 0],
[ 1, 1, 1, ..., 0, 0,
0]], shape=(2, 1000)), 'Confidence', 'Recall']]
```

fitness: 0.6420447839490525

keys: ['metrics/precision(B)', 'metrics/recall(B)', 'metrics/mAP50(B)', 'metrics/mAP50-95(B)']

maps: array([0.71002, 0.57407])

names: {0: 'shoes', 1: 'stamp'}

nt_per_class: array([408, 544])

nt_per_image: array([204, 273])

results_dict: {'metrics/precision(B)': 0.942224418283713, 'metrics/recall(B)': 0.9709686408382363, 'metrics/mAP50(B)': 0.9835409775206505, 'metrics/mAP50-95(B)': 0.6420447839490525, 'fitness': 0.6420447839490525}

save_dir: WindowsPath('C:/Users/LOQ5060/runs/detect/val')

speed: {'preprocess': 1.349584249167229, 'inference': 2.937649084479401, 'loss': 0.0007124537740230232, 'postprocess': 1.8522410255672594}

stats: {'tp': [], 'conf': [], 'pred_cls': [], 'target_cls': [], 'target_img': []}

 **설명** — 학습이 끝난 모델(best.pt)로 검증 데이터(valid 273장)를 평가한다. 학습 때 쓴 data 설정을 기억하고 있어서 인자 없이 호출할 수 있다.

```
In [52]: import pandas as pd

rows = [{"Class": "all", "Precision": metrics.box.mp, "Recall": metrics.box.mr,
        "mAP@.5": metrics.box.map50, "mAP@.5:.95": metrics.box.map}]
for i, c in enumerate(metrics.box.ap_class_index):
    p, r, ap50, ap = metrics.box.class_result(i)
    rows.append({"Class": model.names[int(c)], "Precision": p, "Recall": r, "mAP@.5": ap50, "mAP@.5:.95": ap})
df = pd.DataFrame(rows).round(3)

print("### 📊 Evaluation Results Table (Markdown Format)\n")
print(df.to_markdown(index=False))

print("\n### 📖 Explanation of Each Column")
print("""
- **Class**: Object class name. "all" means aggregated results over all classes.
- **Precision**: Proportion of correct positive predictions (TP / (TP + FP)).
- **Recall**: Proportion of ground truths that were correctly detected (TP / (TP + FN)).
- **mAP@.5**: Mean Average Precision at IoU threshold 0.5.
- **mAP@.5:.95**: mAP averaged over IoU thresholds from 0.5 to 0.95 in steps of 0.05 (more stringent metric).
""")
```

📊 Evaluation Results Table (Markdown Format)

Class	Precision	Recall	mAP@.5	mAP@.5:.95
all	0.942	0.971	0.984	0.642
shoes	0.944	0.944	0.972	0.71
stamp	0.941	0.998	0.995	0.574

📖 Explanation of Each Column

- **Class**: Object class name. "all" means aggregated results over all classes.
- **Precision**: Proportion of correct positive predictions (TP / (TP + FP)).
- **Recall**: Proportion of ground truths that were correctly detected (TP / (TP + FN)).
- **mAP@.5**: Mean Average Precision at IoU threshold 0.5.
- **mAP@.5:.95**: mAP averaged over IoU thresholds from 0.5 to 0.95 in steps of 0.05 (more stringent metric).

 **설명** — 과제 코드는 숫자를 손으로 옮겨 적었지만, 여기서는 `metrics` 객체에서 직접 꺼내 표를 만들었다. 다시 학습해도 표가 자동으로 맞게 바뀐다.

지표	쉽게 말하면
Precision	모델이 "있다"고 한 것 중 진짜 비율 → 낮으면 헛것을 봄(오탐)
Recall	실제로 있는 것 중 찾아낸 비율 → 낮으면 놓침(미탐)
mAP@.5	박스가 절반 이상만 겹쳐도 정답 → "위치를 대충 맞히나"
mAP@.5:.95	0.5~0.95로 점점 엄격하게 → "박스를 딱 맞게 치나"

```
In [53]: metrics.box.map, metrics.box.map50, metrics.box.map75, metrics.box.maps
```

```
Out[53]: (np.float64(0.6420447839490525),
          np.float64(0.9835409775206505),
          np.float64(0.7617668992424378),
          array([ 0.71002, 0.57407]))
```

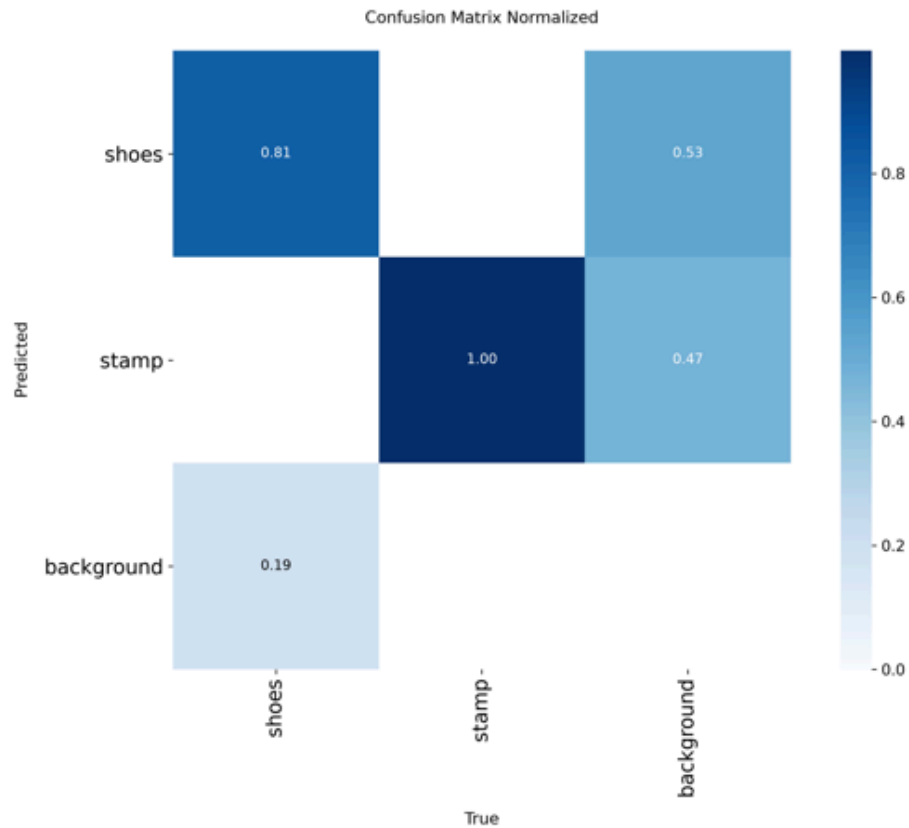
 **설명** — `map` (mAP@.5:.95), `map50`, `map75` (IoU 0.75 기준), `maps` (클래스별 mAP@.5:.95).

평가 결과 해석

	Precision	Recall	mAP@.5	mAP@.5:.95
이번 실행 (RTX 5060, ultralytics 8.4) — all	0.942	0.971	0.984	0.642
shoes	0.944	0.944	0.972	0.710
stamp	0.941	0.998	0.995	0.574
과제 예시 (A100, ultralytics 8.3) — all	0.979	0.903	0.982	0.656

- 환경과 라이브러리 버전이 달라도 **mAP@.5 0.98 / mAP@.5:.95 0.64~0.66** 으로 과제 예시와 거의 같은 수준이 재현됐다.
- **stamp**는 mAP@.5가 0.995로 거의 완벽한데 mAP@.5:.95는 0.574로 shoes(0.710)보다 낮다. 데이터 확인에서 예상한 대로 **작은 도장은 찾기는 잘 찾지만 박스 경계를 정확히 맞추기 어렵다**. mAP75(0.762)가 mAP50(0.984)보다 크게 낮은 것도 같은 이유다.
- **shoes**는 Recall 0.944로 도장보다 더 많이 놓친다.

```
In [54]: display(Image.open(RUNS / "stamp_detect" / "confusion_matrix_normalized.png").resize((640, 480)))
```



설명 — 정규화된 혼동행렬. 세로가 정답, 가로가 예측이다. `background` 행·열은 "물체가 있는데 못 찾음", "없는데 찾음"을 뜻한다.

- **stamp**는 정답의 **100%** 를 stamp로 맞혔다.
- **shoes**는 **81%** 만 맞히고 **19%**는 **배경으로 놓쳤다**(background 행). 신발과 도장을 서로 헷갈린 경우는 없다.
- 오른쪽 background 열은 "실제로는 없는데 찾은 박스"가 어느 클래스로 나왔는지의 비율이다(shoes 53%, stamp 47%). 오탐이 두 클래스에 비슷하게 나뉜다.

→ 이 모델의 약점은 **클래스 혼동이 아니라 신발을 못 찾는 것**이다. 원인은 개인 실험 2-4에서 틀린 사진을 직접 보며 확인한다.

```
In [55]: from PIL import Image
import matplotlib.pyplot as plt

image = Image.open(STAMP / 'test/images/clip1_mp4-33_jpg.rf.95be2fb4fbae44fb12fd600d1d966f76.jpg')

image
```

Out[55]: 2021-10-06 15:17:47
PG03-LASTING-QC



```
In [56]: results = model.predict(str(STAMP / 'test/images/clip1_mp4-33_jpg.rf.95be2fb4fbae44fb12fd600d1d966f76.jpg'))
results[0].save(filename='predicted1.jpg')
```

image 1/1 C:\Users\LOQ5060\Desktop\CV-YOLO\data\stamp\test\images\clip1_mp4-33_jpg.rf.95be2fb4fbae44fb12fd600d1d966f76.jpg: 640x640 2 shoes, 2 stamps, 12.0ms
Speed: 3.0ms preprocess, 12.0ms inference, 2.1ms postprocess per image at shape (1, 3, 640, 640)

Out[56]: 'predicted1.jpg'

```
In [57]: from PIL import Image
import matplotlib.pyplot as plt

image = Image.open("predicted1.jpg")

image
```

Out[57]: 2021-10-06 15:17:47
PG03-LASTING-QC



In [58]: results[0].boxes

Out[58]: ultralytics.engine.results.Boxes object with attributes:

```
cls: tensor([0., 1., 1., 0.], device='cuda:0')
conf: tensor([0.9161, 0.8754, 0.8489, 0.3441], device='cuda:0')
data: tensor([[2.3243e+02, 2.6297e+02, 2.9843e+02, 3.3896e+02, 9.1611e-01, 0.000
0e+00],
            [1.6884e+02, 2.7549e+02, 1.8980e+02, 3.1587e+02, 8.7535e-01, 1.0000e+0
0],
            [2.5227e+02, 9.5641e+01, 2.7378e+02, 1.3422e+02, 8.4886e-01, 1.0000e+0
0],
            [2.4374e+02, 3.6144e+02, 3.1094e+02, 4.1585e+02, 3.4409e-01, 0.0000e+0
0]], device='cuda:0')
id: None
is_track: False
orig_shape: (416, 416)
shape: torch.Size([4, 6])
xywh: tensor([[265.4298, 300.9674, 66.0093, 75.9938],
            [179.3178, 295.6841, 20.9571, 40.3794],
            [263.0239, 114.9320, 21.5084, 38.5821],
            [277.3404, 388.6473, 67.2059, 54.4122]], device='cuda:0')
xywhn: tensor([[0.6381, 0.7235, 0.1587, 0.1827],
            [0.4311, 0.7108, 0.0504, 0.0971],
            [0.6323, 0.2763, 0.0517, 0.0927],
            [0.6667, 0.9342, 0.1616, 0.1308]], device='cuda:0')
xyxy: tensor([[232.4251, 262.9705, 298.4344, 338.9644],
            [168.8393, 275.4944, 189.7964, 315.8738],
            [252.2697, 95.6409, 273.7781, 134.2230],
            [243.7375, 361.4412, 310.9434, 415.8534]], device='cuda:0')
xyxyn: tensor([[0.5587, 0.6321, 0.7174, 0.8148],
            [0.4059, 0.6622, 0.4562, 0.7593],
            [0.6064, 0.2299, 0.6581, 0.3227],
            [0.5859, 0.8688, 0.7475, 0.9996]], device='cuda:0')
```

 **설명** — 학습에 쓰지 않은 **test 이미지**로 예측했다. `results[0].save()` 는 박스를 그린 이미지를 파일로 저장한다. `names` 가 이제 COCO 80개가 아니라 `{0: 'shoes', 1: 'stamp'}` 두 개로 바뀌었다 → 모델이 우리 시나리오에 맞게 바뀌었다는 뜻이다.

- 과제 예시와 같은 test 사진에서 **shoes 2개(0.92, 0.34) + stamp 2개(0.88, 0.85)** 를 찾았다. 과제 예시는 shoes 1 + stamp 2였다.
- 추가로 잡힌 신발(0.34)은 **이미지 아래 가장자리에 잘려 보이는 신발**이다(개인 실험 2-3의 original 그림 참고). 신뢰도가 낮아, 임계값을 0.4로 올리면 과제 예시와 같은 결과가 된다.

개인 실험 2 — 성능 범위를 넓게 확인하기

실험	바꾼 것	확인하려는 것
2-1	모델·에폭 (YOLOv8n 20/50, YOLO11n , YOLOv8s)	오래 학습하면? 최신 모델이면? 큰 모델이면?
2-2	신뢰도 임계값	현장 운영 기준(농침 vs 헛것) 정하기
2-3	추가 샘플 데이터 : test 193장 전체 + 조건 변화(어둡게·흐리게·노이즈)	처음 보는 데이터와 나쁜 촬영 환경에서도 버티나?
2-4	틀린 사례 모아보기	어떤 상황에서 틀리나?

모든 비교는 학습에 쓰지 않은 **test** 셋 기준이다.

```
In [59]: EXPS = [("stamp_detect", "yolov8n.pt", 20), ("v8n_e50", "yolov8n.pt", 50),
               ("11n_e20", "yolo11n.pt", 20), ("v8s_e20", "yolov8s.pt", 20)]
summary = []
for name, weights, ep in EXPS:
    run = RUNS / name
    if not (run / "weights" / "best.pt").exists(): # 이미 학습한 실험은 결과
                                                    # 재사용
        YOLO(weights).train(data=str(DATA), epochs=ep, imgsz=640, batch=16, device=DEVICE,
                             project=str(RUNS), name=name, exist_ok=True, workers
                             =2, seed=42, verbose=False)
        m = YOLO(str(run / "weights" / "best.pt"))
        v = m.val(data=str(DATA), split="test", device=DEVICE, verbose=False, plots
                  =False,
                  project=str(RUNS), name=f"{name}_test", exist_ok=True)
        res = pd.read_csv(run / "results.csv")
        summary.append({"exp": name, "model": weights, "epochs": ep, "best_epoch":
            int(res.loc[res["metrics/mAP50-95(B)"].idxmax(), "epoch"]),
            "params(M)": round(sum(p.numel() for p in m.model.parameters
            ()) / 1e6, 2),
            "P": v.box.mp, "R": v.box.mr, "mAP50": v.box.map50, "mAP50-
            95": v.box.map,
            "shoes mAP50-95": v.box.maps[0], "stamp mAP50-95": v.box.map
            s[1],
            "infer_ms": v.speed["inference"]})
exp_df = pd.DataFrame(summary).round(3)
(ROOT / "results").mkdir(exist_ok=True)
exp_df.to_csv(ROOT / "results" / "stamp_experiments.csv", index=False)
exp_df
```

Ultralytics 8.4.154 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU, 8151MiB)

Model summary (fused): 72 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs

val: Fast image access (ping: 0.10.0 ms, read: 131.733.9 MB/s, size: 27.8 KB)

val: Scanning C:\Users\LQ5060\Desktop\CV-YOLO\data\stamp\test\labels.cache... 193 images, 0 backgrounds, 0 corrupt: 100% 193/193 0.0s

all 193 633 0.946 0.973 0.984
0.637

Speed: 1.3ms preprocess, 2.8ms inference, 0.0ms loss, 1.0ms postprocess per image

Ultralytics 8.4.154 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU, 8151MiB)

Model summary (fused): 72 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs

val: Fast image access (ping: 0.10.0 ms, read: 119.413.7 MB/s, size: 26.7 KB)

val: Scanning C:\Users\LQ5060\Desktop\CV-YOLO\data\stamp\test\labels.cache... 193 images, 0 backgrounds, 0 corrupt: 100% 193/193 0.0s

all 193 633 0.982 0.92 0.979
0.626

... (2줄 생략) ...

YOLO11n summary (fused): 100 layers, 2,582,542 parameters, 0 gradients, 6.4 GFLOPs

val: Fast image access (ping: 0.10.0 ms, read: 175.735.4 MB/s, size: 27.6 KB)

val: Scanning C:\Users\LQ5060\Desktop\CV-YOLO\data\stamp\test\labels.cache... 193 images, 0 backgrounds, 0 corrupt: 100% 193/193 0.0s

all 193 633 0.975 0.931 0.982
0.634

Speed: 1.0ms preprocess, 3.1ms inference, 0.0ms loss, 1.0ms postprocess per image

Ultralytics 8.4.154 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU, 8151MiB)

Model summary (fused): 72 layers, 11,126,358 parameters, 0 gradients, 28.4 GFLOPs

val: Fast image access (ping: 0.10.0 ms, read: 166.432.9 MB/s, size: 27.1 KB)

val: Scanning C:\Users\LQ5060\Desktop\CV-YOLO\data\stamp\test\labels.cache... 193 images, 0 backgrounds, 0 corrupt: 100% 193/193 0.0s

all 193 633 0.962 0.939 0.975
0.628

Speed: 1.1ms preprocess, 6.6ms inference, 0.0ms loss, 0.9ms postprocess per image

Out[59]:

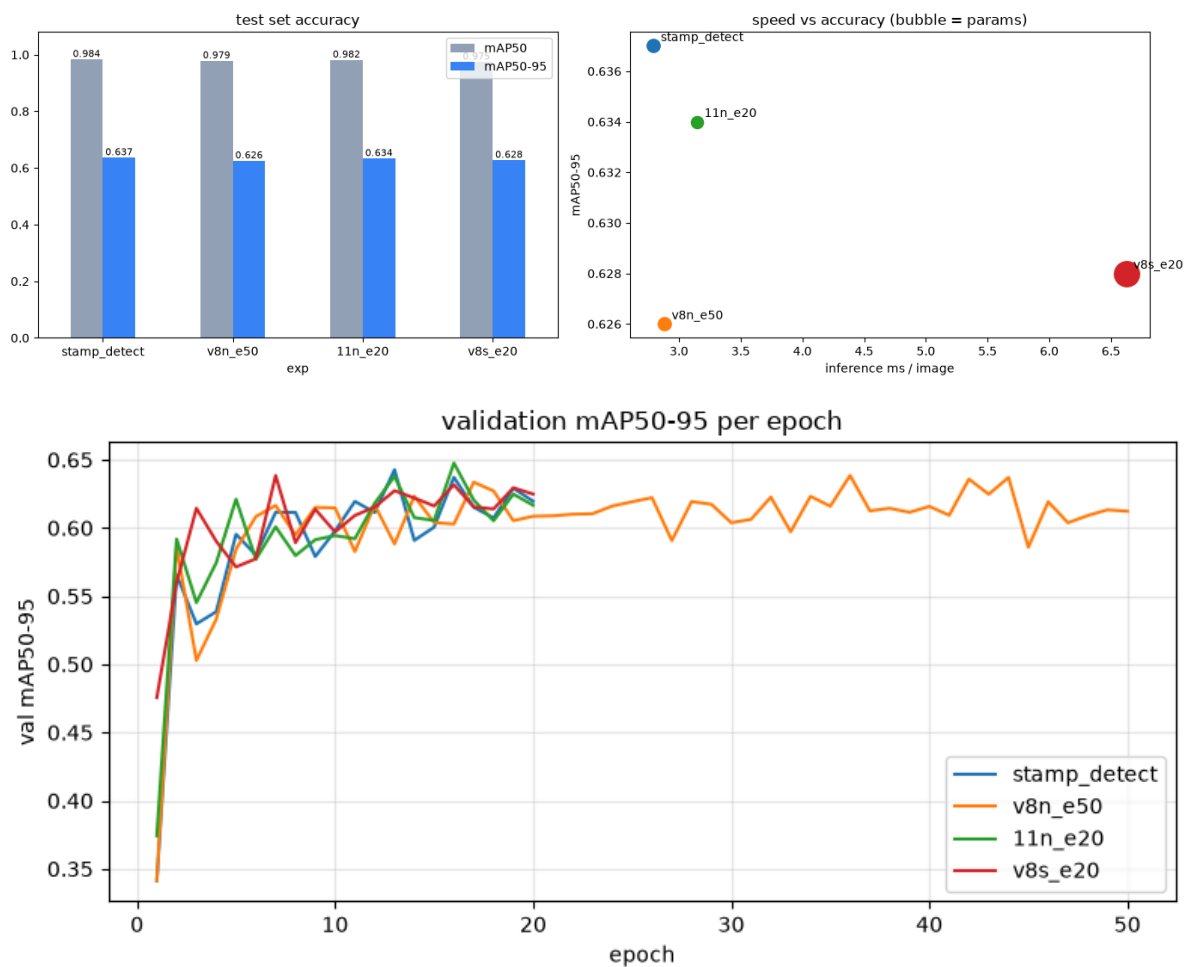
	exp	model	epochs	best_epoch	params(M)	P	R	mAP50	mAI
0	stamp_detect	yolov8n.pt	20	13	3.01	0.946	0.973	0.984	0
1	v8n_e50	yolov8n.pt	50	36	3.01	0.982	0.920	0.979	0
2	11n_e20	yolo11n.pt	20	16	2.59	0.975	0.931	0.982	0
3	v8s_e20	yolov8s.pt	20	7	11.14	0.962	0.939	0.975	0

```

In [60]: fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))
exp_df.plot.bar(x="exp", y=["mAP50", "mAP50-95"], rot=0, ax=axes[0], color=["#9
4a3b8", "#3b82f6"])
axes[0].set_ylim(0, 1.08); axes[0].set_title("test set accuracy")
for c in axes[0].containers: axes[0].bar_label(c, fmt="%.3f", fontsize=8)
for _, r in exp_df.iterrows():
    axes[1].scatter(r["infer_ms"], r["mAP50-95"], s=r["params(M)"] * 40)
    axes[1].annotate(r["exp"], (r["infer_ms"], r["mAP50-95"]), textcoords="offse
t points", xytext=(6, 4))
axes[1].set_xlabel("inference ms / image"); axes[1].set_ylabel("mAP50-95"); axes
[1].set_title("speed vs accuracy (bubble = params)")
plt.tight_layout(); plt.savefig(ROOT / "report" / "images" / "stamp_experiments.
png", dpi=150); plt.show()

# 에폭별 검증 mAP 곡선
plt.figure(figsize=(8, 4))
for name, _, _ in EXPS:
    res = pd.read_csv(RUNS / name / "results.csv")
    plt.plot(res["epoch"], res["metrics/mAP50-95(B)"], label=name)
plt.xlabel("epoch"); plt.ylabel("val mAP50-95"); plt.legend(); plt.title("valida
tion mAP50-95 per epoch"); plt.grid(alpha=.3)
plt.tight_layout(); plt.savefig(ROOT / "report" / "images" / "stamp_epoch_curve
s.png", dpi=150); plt.show()

```



실험 2-1 결과 해석

실험	파라미터	test mAP50	test mAP50-95	shoes	stamp	추론(ms/장)
YOLOv8n · 20ep (기본)	3.01M	0.984	0.637	0.702	0.572	2.8
YOLOv8n · 50ep	3.01M	0.979	0.626	0.700	0.553	2.9
YOLO11n · 20ep	2.59M	0.982	0.634	0.692	0.576	3.1
YOLOv8s · 20ep	11.14M	0.975	0.628	0.693	0.564	6.6

- 네 조합 모두 **test mAP50-95가 0.626~0.637로 거의 같다.** 차이(0.011)는 에폭마다 오르내리는 폭(0.58~0.64)보다 작다.
- **50 에폭:** 최고점은 36 에폭이었지만 test 성능은 오히려 조금 낮았다. 곡선을 보면 20 에폭 이후 0.60~0.64에서 오르내리기만 한다 → **더 오래 학습해도 이득이 없다.**
- **YOLOv8s:** 파라미터 3.7배, 추론 시간 2.4배(2.8 → 6.6ms)인데 정확도는 오르지 않았다 → 이 데이터에서는 **작은 모델로 충분하다.**
- **YOLO11n(다른 모델):** 파라미터 14% 적고 정확도는 같은 수준이며, stamp는 가장 높았다. 현장에 올린다면 v8n이나 11n이 합리적이다.
- 모든 모델에서 **stamp mAP50-95(0.55~0.58)가 shoes(0.69~0.70)보다 낮다.** 모델을 바꿔도 해결되지 않으므로, 한계는 모델이 아니라 **작은 물체의 박스 정밀도와 라벨 경계**에 있다고 판단했다. 개선하려면 입력 해상도를 키우거나(원본이 416이라 효과는 제한적) 라벨 품질을 높여야 한다.

```
In [61]: best_name = exp_df.sort_values("mAP50-95", ascending=False).iloc[0]["exp"]
best_model = YOLO(str(RUNS / best_name / "weights" / "best.pt"))
print("가장 좋은 모델:", best_name)

rows = []
for c in [0.05, 0.1, 0.25, 0.4, 0.5, 0.6, 0.75, 0.9]:
    v = best_model.val(data=str(DATA), split="test", conf=c, device=DEVICE, verbose=False, plots=False,
                       project=str(RUNS), name="conf_sweep", exist_ok=True)
    for i, k in enumerate(v.box.ap_class_index):
        p, r, *_ = v.box.class_result(i)
        rows.append({"conf": c, "class": best_model.names[int(k)], "precision": p, "recall": r})
sweep = pd.DataFrame(rows)
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, cls in zip(axes, ["shoes", "stamp"]):
    s = sweep[sweep["class"] == cls]
    ax.plot(s.conf, s.precision, "o-", label="precision"); ax.plot(s.conf, s.recall, "o-", label="recall")
    ax.set_title(cls); ax.set_xlabel("conf threshold"); ax.set_ylim(0, 1.05); ax.grid(alpha=.3); ax.legend()
plt.tight_layout(); plt.savefig(ROOT / "report" / "images" / "stamp_conf_sweep.png", dpi=150); plt.show()
sweep.pivot(index="conf", columns="class", values=["precision", "recall"]).round(3)
```

가장 좋은 모델: stamp_detect

Ultralytics 8.4.154 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU, 8151MiB)

Model summary (fused): 72 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs

val: Fast image access (ping: 0.10.0 ms, read: 126.731.2 MB/s, size: 27.3 KB)

val: Scanning C:\Users\LOQ5060\Desktop\CV-YOLO\data\stamp\test\labels.cache... 193 images, 0 backgrounds, 0 corrupt: 100% 193/193 0.0s

	all	193	633	0.944	0.975	0.98
--	-----	-----	-----	-------	-------	------

0.631

Speed: 1.2ms preprocess, 2.9ms inference, 0.0ms loss, 0.9ms postprocess per image

Ultralytics 8.4.154 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU, 8151MiB)

Model summary (fused): 72 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs

val: Fast image access (ping: 0.10.0 ms, read: 125.130.7 MB/s, size: 27.5 KB)

val: Scanning C:\Users\LOQ5060\Desktop\CV-YOLO\data\stamp\test\labels.cache... 193 images, 0 backgrounds, 0 corrupt: 100% 193/193 0.0s

... (35줄 생략) ...

Model summary (fused): 72 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs

val: Fast image access (ping: 0.10.0 ms, read: 171.528.1 MB/s, size: 27.1 KB)

val: Scanning C:\Users\LOQ5060\Desktop\CV-YOLO\data\stamp\test\labels.cache... 193 images, 0 backgrounds, 0 corrupt: 100% 193/193 0.0s

	all	193	633	0.996	0.884	0.884
--	-----	-----	-----	-------	-------	-------

0.569

Speed: 1.3ms preprocess, 3.3ms inference, 0.0ms loss, 1.0ms postprocess per image

Ultralytics 8.4.154 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU, 8151MiB)

Model summary (fused): 72 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs

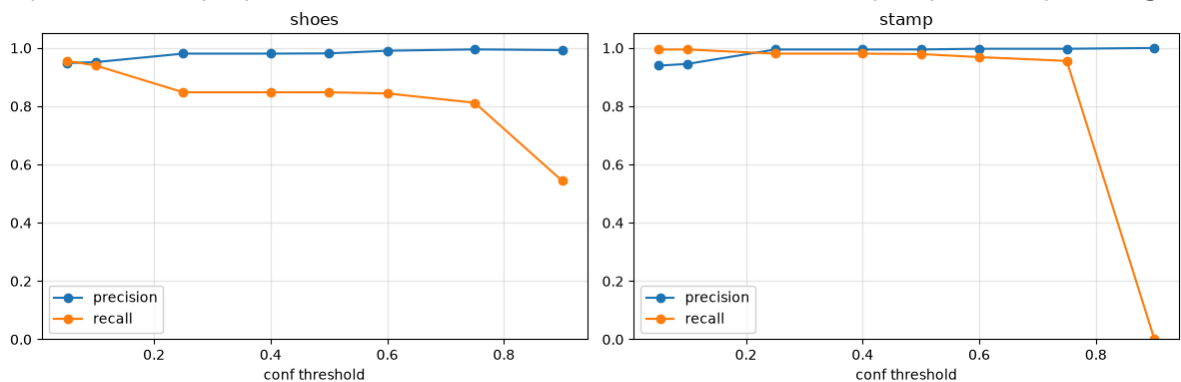
val: Fast image access (ping: 0.10.0 ms, read: 123.626.1 MB/s, size: 27.7 KB)

val: Scanning C:\Users\LOQ5060\Desktop\CV-YOLO\data\stamp\test\labels.cache... 193 images, 0 backgrounds, 0 corrupt: 100% 193/193 0.0s

	all	193	633	0.996	0.273	0.274
--	-----	-----	-----	-------	-------	-------

0.201

Speed: 1.3ms preprocess, 3.1ms inference, 0.0ms loss, 0.6ms postprocess per image



Out[61]:

class	precision		recall	
	shoes	stamp	shoes	stamp
	conf			
0.05	0.948	0.939	0.956	0.995
0.10	0.951	0.945	0.940	0.995
0.25	0.981	0.995	0.848	0.981
0.40	0.981	0.995	0.848	0.981
0.50	0.981	0.995	0.848	0.979
0.60	0.991	0.997	0.844	0.969
0.75	0.995	0.997	0.812	0.956
0.90	0.993	1.000	0.544	0.003

 **설명** — 임계값을 올리면 확실한 박스만 남아 Precision은 오르고 Recall은 떨어진다. 이 곡선으로 현장 운영 기준을 정할 수 있다. (`model.val()` 은 원래 `conf=0.001`로 평가해서 mAP를 계산하므로, 여기서는 P-R 변화만 본다.)

실험 2-2 결과 해석

가장 좋은 모델(YOLOv8n 20ep)로 test 셋을 평가했다.

- **stamp:** 임계값 0.05~0.75에서 Recall이 0.995 → 0.956으로 거의 유지되고 Precision은 0.94 → 0.997로 오른다. 그런데 **0.9에서는 Recall이 0.003**으로 무너진다. 도장 예측의 신뢰도가 대부분 0.85 안팎이라 0.9를 넘는 박스가 거의 없기 때문이다.
- **shoes:** 0.10까지는 Recall 0.94인데 **0.25부터 0.848로 뚝 떨어진다**. 신발 예측 중에 신뢰도 0.1~0.25 사이의 애매한 박스(가려진 신발, 가장자리 신발)가 많다는 뜻이다.

→ 운영 기준 제안

- 공정에서 **신발을 놓치면 안 된다면 conf 0.1** (shoes P 0.951 / R 0.940, stamp P 0.945 / R 0.995)
- **오탐을 줄이고 싶다면 0.25~0.5** (Precision 0.98 이상)
- 클래스마다 신뢰도 분포가 다르므로 **클래스별로 임계값을 따로 두는 것**이 더 낫다.

In [62]: `import shutil`

```
def _dark(im):    return cv2.convertScaleAbs(im, alpha=0.45, beta=0)    # 조명 부족
def _bright(im): return cv2.convertScaleAbs(im, alpha=1.3, beta=60)    # 과노출
def _blur(im):    return cv2.GaussianBlur(im, (9, 9), 0)              # 초점 흐림 / 움직임
def _noise(im):
    n = np.random.default_rng(0).normal(0, 25, im.shape)              # 센서 노이즈
    return np.clip(im.astype(np.float32) + n, 0, 255).astype(np.uint8)

CONDS = {"original": None, "dark": _dark, "bright": _bright, "blur": _blur, "noise": _noise}
rows, examples = [], {}
ex_img = STAMP / "test/images/clip1_mp4-33_jpg.rf.95be2fb4fbae44fb12fd600d1d966f76.jpg"
for cond, fn in CONDS.items():
    d = ROOT / "data" / "stamp_robust" / cond
    if not (d / "data.yaml").exists():
        (d / "test" / "images").mkdir(parents=True, exist_ok=True)
        shutil.copypath(STAMP / "test" / "labels", d / "test" / "labels", dirs_exist_ok=True)
        for p in (STAMP / "test" / "images").glob("*"):
            im = cv2.imread(str(p)); cv2.imwrite(str(d / "test" / "images" / p.name), fn(im) if fn else im)
            (d / "data.yaml").write_text(yaml.safe_dump({"path": d.as_posix(), "train": "test/images", "val": "test/images", "test": "test/images", "names": cfg["names"]}), encoding="utf-8")
            v = best_model.val(data=str(d / "data.yaml"), split="test", device=DEVICE, verbose=False, plots=False, project=str(RUNS), name="robust", exist_ok=True)
            rows.append({"condition": cond, "mAP50": v.box.map50, "mAP50-95": v.box.map, "shoes R": v.box.class_result(0)[1], "stamp R": v.box.class_result(1)[1]})
        examples[cond] = best_model.predict(str(d / "test" / "images" / ex_img.name), device=DEVICE, verbose=False)[0].plot()[ :, :, :-1]
    robust = pd.DataFrame(rows).round(3)
    robust.to_csv(ROOT / "results" / "stamp_robustness.csv", index=False)

fig, axes = plt.subplots(1, 5, figsize=(22, 4.5))
for ax, (cond, im) in zip(axes, examples.items()):
    ax.imshow(im); ax.set_title(cond); ax.axis("off")
plt.tight_layout(); plt.savefig(ROOT / "report" / "images" / "stamp_robust_examples.png", dpi=110); plt.show()
robust
```

Ultralytics 8.4.154 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU, 8151MiB)

Model summary (fused): 72 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs
WARNING **val:** Slow image access detected (ping: 0.10.1 ms, read: 6.50.8 MB/s, size: 50.0 KB). Use local storage instead of remote/mounted storage for better performance. See <https://docs.ultralytics.com/guides/model-training-tips>

val: New cache created: C:\Users\LOQ5060\Desktop\CV-YOLO\data\stamp_robust\original\test\labels.cache

	all	193	633	0.947	0.973	0.984
--	-----	-----	-----	-------	-------	-------

0.635

Speed: 1.3ms preprocess, 2.5ms inference, 0.0ms loss, 1.0ms postprocess per image

Ultralytics 8.4.154 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU, 8151MiB)

Model summary (fused): 72 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs
WARNING **val:** Slow image access detected (ping: 0.20.1 ms, read: 3.90.5 MB/s, size: 38.5 KB). Use local storage instead of remote/mounted storage for better performance. See <https://docs.ultralytics.com/guides/model-training-tips>

val: New cache created: C:\Users\LOQ5060\Desktop\CV-YOLO\data\stamp_robust\dark\test\labels.cache

	all	193	633	0.967	0.964	0.983
--	-----	-----	-----	-------	-------	-------

0.637

... (10줄 생략) ...

Model summary (fused): 72 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs
WARNING **val:** Slow image access detected (ping: 0.10.0 ms, read: 3.80.5 MB/s, size: 37.0 KB). Use local storage instead of remote/mounted storage for better performance. See <https://docs.ultralytics.com/guides/model-training-tips>

val: New cache created: C:\Users\LOQ5060\Desktop\CV-YOLO\data\stamp_robust\blur\test\labels.cache

	all	193	633	0.973	0.915	0.976
--	-----	-----	-----	-------	-------	-------

0.548

Speed: 1.5ms preprocess, 4.4ms inference, 0.0ms loss, 1.4ms postprocess per image

Ultralytics 8.4.154 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU, 8151MiB)

Model summary (fused): 72 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs
WARNING **val:** Slow image access detected (ping: 0.20.0 ms, read: 12.51.3 MB/s, size: 138.7 KB). Use local storage instead of remote/mounted storage for better performance. See <https://docs.ultralytics.com/guides/model-training-tips>

val: New cache created: C:\Users\LOQ5060\Desktop\CV-YOLO\data\stamp_robust\noise\test\labels.cache

	all	193	633	0.981	0.587	0.777
--	-----	-----	-----	-------	-------	-------

0.478

Speed: 1.7ms preprocess, 5.5ms inference, 0.0ms loss, 1.4ms postprocess per image



Out[62]:

	condition	mAP50	mAP50-95	shoes R	stamp R
0	original	0.984	0.635	0.952	0.995
1	dark	0.983	0.637	0.938	0.989
2	bright	0.980	0.640	0.922	0.987
3	blur	0.976	0.548	0.844	0.987
4	noise	0.777	0.478	0.223	0.950

실험 2-3 결과 해석

조건	mAP50	mAP50-95	shoes Recall	stamp Recall
original	0.984	0.635	0.952	0.995
dark (밝기 45%)	0.983	0.637	0.938	0.989
bright (과노출)	0.980	0.640	0.922	0.987
blur (흐림)	0.976	0.548	0.844	0.987
noise (센서 노이즈)	0.777	0.478	0.223	0.950

- **조명 변화(dark/bright)** 에는 거의 영향이 없다. 학습 때 Ultralytics가 기본으로 밝기·채도를 흔드는 증강(`hsv_v=0.4` 등)을 적용했기 때문으로 보인다.
- **흐림**은 mAP50은 유지되지만 mAP50-95가 0.09 떨어졌다. 대충은 찾지만 경계가 뭉개져 박스가 덜 정확해진다.
- **노이즈**에서는 **신발 Recall이 0.95 → 0.22로 급락**했다. 반면 도장은 0.95로 버텼다. 예시 그림에서도 노이즈 조건에서 아래쪽 신발을 놓쳤다. 어두운 신발의 무늬가 노이즈에 묻히는 반면, 도장은 흰 작업대 위에서 색 대비가 커서 버티는 것으로 보인다.

→ 현장 적용 시에는 **조명을 충분히 확보해 센서 노이즈(카메라 게인)를 줄이는 것**이 가장 중요하다. 학습 데이터에 **노이즈·블러 증강**을 추가하면 개선될 것으로 기대한다.

```

In [63]: # test 셋에서 틀린 사례 찾기: 정답 개수와 예측 개수가 다른 이미지
def count_gt(p):
    lines = [l for l in (STAMP / "test" / "labels" / (p.stem + ".txt")).read_text().splitlines() if l.strip()]
    c = [int(l.split()[0]) for l in lines]
    return c.count(0), c.count(1)

wrong = []
for r in best_model.predict([str(p) for p in sorted((STAMP / "test" / "images").glob("*"))], conf=0.25,
                             device=DEVICE, stream=True, verbose=False):
    p = Path(r.path); gs, gt = count_gt(p)
    pc = r.bboxes.cls.tolist(); ps, pt = pc.count(0), pc.count(1)
    if (gs, gt) != (ps, pt):
        wrong.append({"image": p.name, "gt shoes": gs, "pred shoes": ps, "gt stamp": gt, "pred stamp": pt, "_plot": r.plot()[ :, :, :-1]})
print(f"개수가 틀린 이미지: {len(wrong)} / 193")
wrong_df = pd.DataFrame(wrong).drop(columns="_plot")
display(wrong_df.head(10))
if wrong:
    k = min(6, len(wrong))
    fig, axes = plt.subplots(1, k, figsize=(4 * k, 4), squeeze=False)
    for ax, w in zip(axes[0], wrong[:k]):
        ax.imshow(w["_plot"]); ax.set_title(f"GT s{w['gt shoes']}/t{w['gt stamp']} pred s{w['pred shoes']}/t{w['pred stamp']}", fontsize=9); ax.axis("off")
    plt.tight_layout(); plt.savefig(ROOT / "report" / "images" / "stamp_errors.png", dpi=110); plt.show()

```

개수가 틀린 이미지: 40 / 193

	image	gt shoes	pred shoes	gt stamp	pred stamp
0	clip1_mp4-1075_jpg.rf.57825344122889ed12f7a9d0...	4	4	1	2
1	clip1_mp4-1076_jpg.rf.eb74636993a043016bd5feee...	4	4	2	4
2	clip1_mp4-1092_jpg.rf.a4d3481c456a22d664f8df3c...	3	3	2	5
3	clip1_mp4-149_jpg.rf.4463b36f6d52e4b5a91b9159f...	2	1	2	2
4	clip1_mp4-157_jpg.rf.a09074ebd2a276b0ae3a852d6...	2	1	2	3
5	clip1_mp4-227_jpg.rf.baa51515a208848a50a6e30ab...	1	0	2	2
6	clip1_mp4-233_jpg.rf.790d2884ae480e950f888ad30...	2	1	2	2
7	clip1_mp4-245_jpg.rf.0c5b80fed0f649e2929489009...	2	1	2	2
8	clip1_mp4-247_jpg.rf.cbac2e53983e7abf1eded5e98...	2	1	2	2
9	clip1_mp4-255_jpg.rf.aae09c7c0e326fac0d14c593d...	2	1	2	2



실험 2-4 결과 해석

conf 0.25 기준으로 **193장 중 40장(21%)** 에서 정답 개수와 예측 개수가 달랐다. 틀린 사진을 직접 보면 세 가지 유형이 보인다.

1. 작업자가 손에 들고 있는 신발을 놓침 (정답 신발 2 → 예측 1): 신발이 손·팔에 가려지고 작업대 위의 신발과 모양·각도가 달라서 못 찾는다. 혼동행렬에서 본 "shoes 19% 놓침"의 주 원인이다.
2. 도장 하나에 박스가 두 개 (정답 도장 2 → 예측 4~5): 겹친 박스의 신뢰도가 0.51/0.52로 비슷해 NMS로 지워지지 않았다.
3. 신발 위의 작은 부분을 도장으로 착각: 신발 안쪽에 신뢰도 0.28의 stamp 박스가 생겼다.

→ 개선 방향: 손에 든 신발·가려진 신발 사진을 더 모아 학습하고, 도장 중복은 NMS IoU를 조금 낮추거나 도장만 conf를 0.5 이상으로 올리면 줄어든 것이다.

목표 기준 평가 및 성과

목표	수행 내용	결과
① 추론 결과 구조 이해	<code>Results / Boxes</code> 구조, <code>xywh·xyxy</code> 정규화 좌표를 직접 계산해 검증, 클래스 필터링, crop	직접 계산한 값과 <code>xywhn / xyxyn</code> 이 일치함을 확인
② 제조 데이터 학습·평가	stamp v10으로 YOLOv8n 20 에폭 파인튜닝, 평가표 자동 생성	val mAP@.5 0.984 / mAP@.5:.95 0.642 (과제 예시 0.982 / 0.656과 같은 수준)
③ 성능 범위 실험	임계값·NMS·추가 샘플·모델 3종 (YOLO11n 포함)·에폭·촬영 조건 5종·오류 사례 분석	모델·에폭보다 데이터(작은 도장, 가려진 신발)와 촬영 환경(노이즈) 이 성능을 결정함
(추가) 레포 리팩터링	노트북 코드를 <code>src/ · tools/</code> 스크립트로 분리, README 작성, 제출용 PDF 자동 생성	https://github.com/hwang-ye-song/CV_YOLO

성과 요약

- 20 에폭(약 3.5분)의 가벼운 학습만으로 도장 검출 **Recall 0.998**. "도장이 찍혔는지"를 확인하는 공정 목적에는 충분한 수준이다.
- 약점은 **가려진 신발 미탐(19%)** 과 **노이즈 환경에서의 급격한 성능 저하**이며, 원인과 개선 방향(데이터 추가, 증강, 조명·클래스별 임계값)을 실험으로 찾아냈다.

개인 회고

KPT

- **Keep**
 - 기본 코드를 그대로 따라가며 좌표 값을 **직접 계산해 확인**한 것. 라이브러리 출력의 의미를 확실히 이해할 수 있었다.
 - 결과 숫자를 손으로 옮기지 않고 `metrics` 객체에서 **자동으로 표를 만든** 것. 재실행해도 표가 틀리지 않는다.
 - 모델 하나의 점수에서 멈추지 않고 **왜 그 점수가 나오는지**(작은 물체, 가려짐, 노이즈)까지 실험으로 확인한 것.
- **Problem**
 - Roboflow 링크가 다른 버전(v2, 이미지 3장)을 받아와 한동안 헤맸다. 데이터셋 **버전과 분할 수를 먼저 확인**했어야 했다.
 - Roboflow의 `data.yaml` 이 상대경로라 실행 위치에 따라 경로가 깨졌다.
 - 과제 코드의 결과 경로(`runs/detect/predict`)를 고정하면, 여러 번 실행할 때 `predict2`, `predict3` 로 바뀌어 이미지를 못 찾는다.
- **Try**
 - 학습 데이터에 **노이즈·블러 증강**을 넣고 test 조건 실험을 다시 해서 개선 폭을 확인한다.
 - 클래스별로 다른 신뢰도 임계값을 적용하는 후처리를 만든다.
 - 직접 촬영한 제품 사진으로 **정상/불량 영역 탐지**를 학습해 실제 현장 적용 가능성을 검증한다 (다음 프로젝트).

AAR (After Action Review)

질문	답
원래 의도한 것은?	과제 코드를 재현하고, 모델을 바꿔 성능을 끌어올린다
실제로 일어난 일은?	재현은 성공했지만(mAP50 0.984), 모델·에폭을 바꿔도 성능이 오르지 않았다
차이가 생긴 이유는?	성능 한계가 모델 용량이 아니라 데이터 특성 (작은 도장의 박스 정밀도, 손에 가려진 신발)에 있었다
다음에 할 것은?	모델 크기보다 데이터 보강·증강과 촬영 환경 개선 에 먼저 시간을 쓴다. 성능이 멈추면 틀린 사례부터 본다

동료 피드백 (선택)

- (받은 피드백을 여기에 기록)